18-740: Modern Computer Architecture & Design

Lecture 11: "Operating System & Runtime Environment"

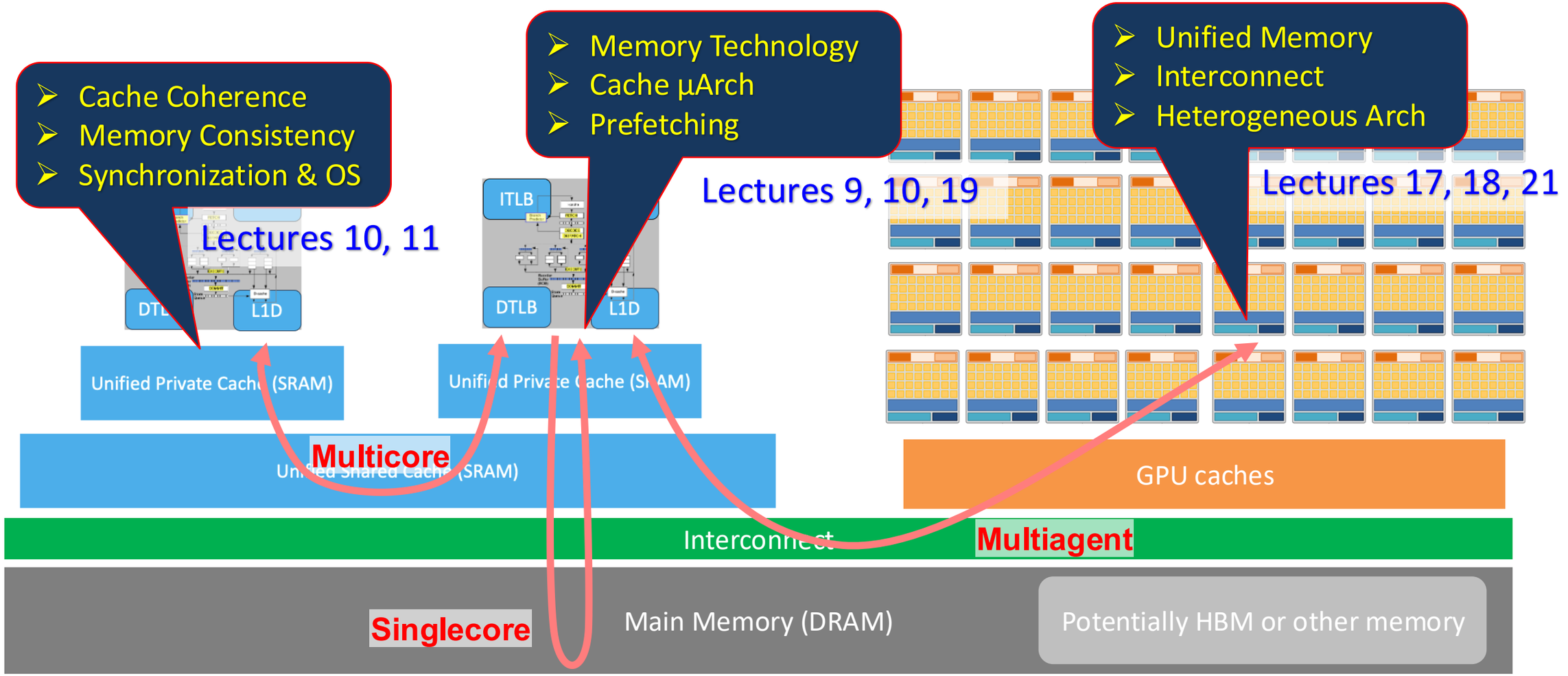
Scott Mionis

September 30, 2025

- From **Synthesis Lectures on Computer Architecture**:
- Core Reading:
  - Sec. 5: Bhattacharjee et al, "Architectural and Operating System Support for Virtual Memory," 2017
  - Skim: Scott Michael L., "Shared Memory Synchronization", 2013
- Elective Reading:
  - Sec. 3: Bugnion et al, "Hardware and Software Support for Virtualization", 2017

Electrical & Computer ENGINEERING

Memory Data Flow : Roadmap

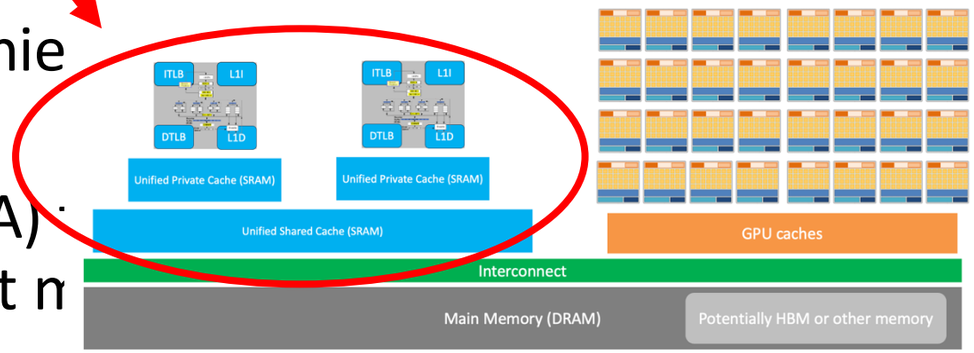


Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Recall *hardware* support for shared memory

We built what is typically referred to as a "Shared Memory Multiprocessor" (MIMD)

- ***Cache Coherence*** allows an arbitrary cache hie
  contiguous memory array (software view)
- ***Memory Consistency*** is the specification (ISA)
  can get legally reordered in shared, coherent m



We are left with a very complicated machine to operate…

Page Tables, Interrupt Handlers, I$ and TLB Management, Exception Handlers, All Input/Output, Timesharing, …

The scaffolding required to actually make such a machine usable is complex!

Primordial bootstrapping questions

- *How do instructions get into memory?*
  - *if all programs must be loaded from a file (.exe, .o, .elf, .out), but loading said file requires a program…*
- *How do page tables get into memory?*
  - *if you can't access memory without page tables, but must write page tables to memory…*
- *How is it possible to ever run more than one thing concurrently?*
  - *740 CPU has only one PC, one register file…*

The next two lectures are about *Threads*

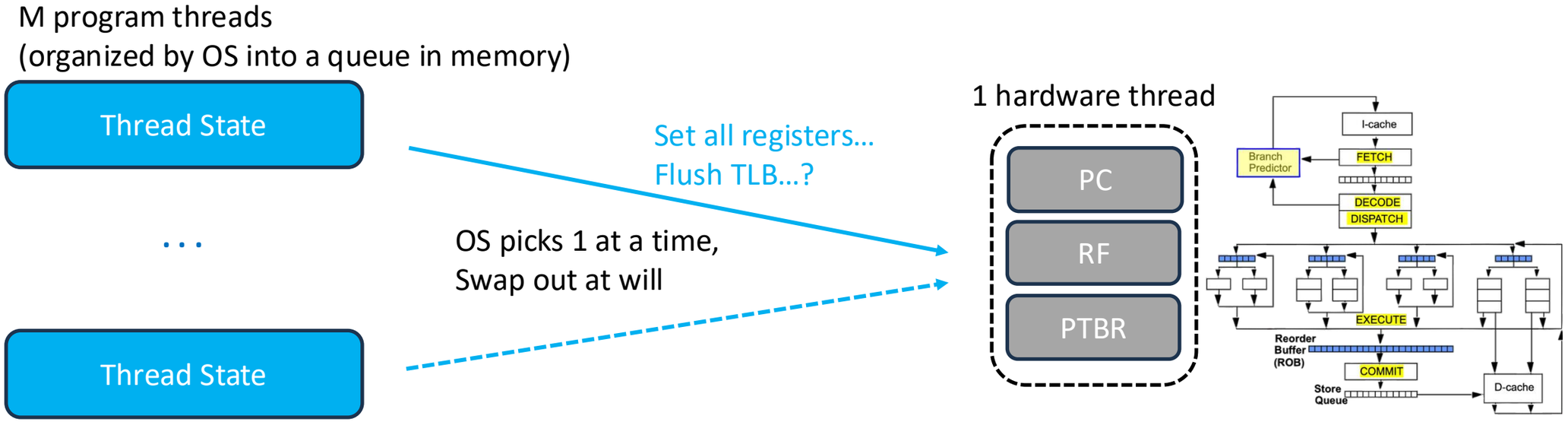
I.e. the fundamental execution of work in a Von-Neumann machine

- ***An extremely abstract concept***
  - A “schedulable entity” or “thing you can run”
  - GPU/CPU differ radically in how they represent work
- A program generally ***has*** at least one thread
  - A multithreaded program is explicitly annotated (pthreads aka `clone’) to spawn more
- A CPU generally can ***run*** at least one thread
  - A multithreaded CPU can run multiple concurrently (duplicate PC/RF)

The next two lectures are about *Threads*

I.e. the fundamental execution of work in a Von-Neumann machine

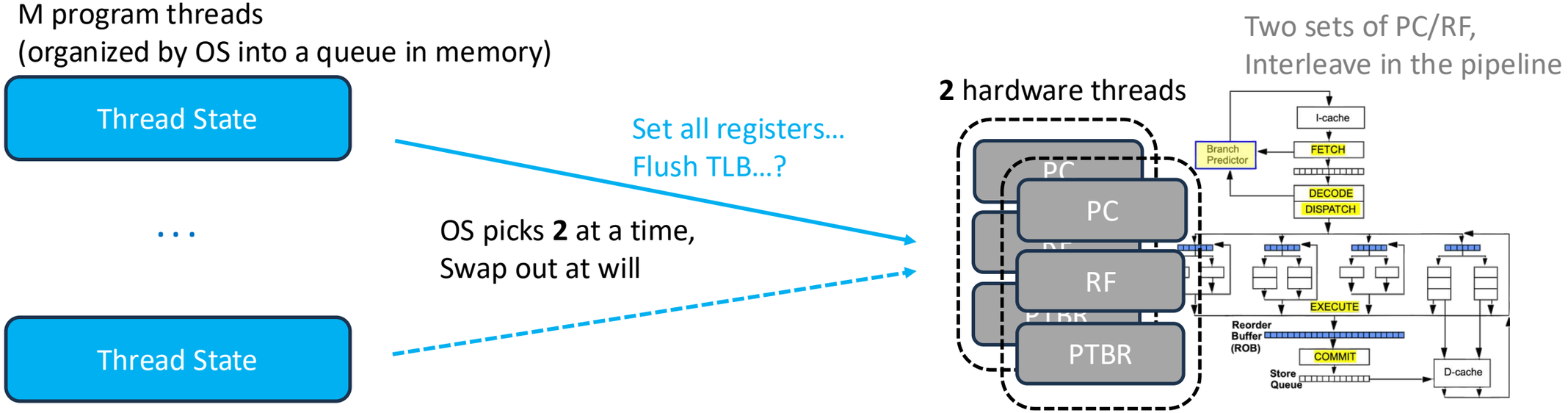
- ***Software Threads***
  - Time-multiplexed in OS (i.e. context switch routine)



The next two lectures are about *Threads*

I.e. the fundamental execution of work in a Von-Neumann machine

- ***Software Threads***
  - Time-multiplexed in OS (i.e. context switch routine)
- ***Hardware Threads***
  - Time-multiplexed in hardware



Origins of timesharing

- Earliest computers just took a program and executed it end-to-end
  - No concept of software (or hardware) threads. *Programs simply a list of instructions*
  - Programs (unless human operator pressed a stop button) ran to completion
- Memory barely large enough for a single program
  - Let alone *multiple* at a time
  - Let alone a management program to run programs (OS)

Origins of timesharing

“Context Switch”
(put cards in reader, press buttons)



Source: Wikipedia – IBM 1401, Computer History Museum

“Runnable Queue”

Origins of timesharing

- Logical progression from
- ***Instead of loading one pr***
  - Requires enough memory
- ***With multiple programs***
  - Requires piggy-backing on
    and into whatever this "pr
- ***With code to pick/run, co***
  - Make sure a short prograr
  - Be fair to all users of the c
  - Switch program out when

"Context Switch"
(put cards in reader, press buttons)

"Runnable Queue"

Source: Wikipedia – IBM 1401, Computer History Museum

Thread-level-parallelism

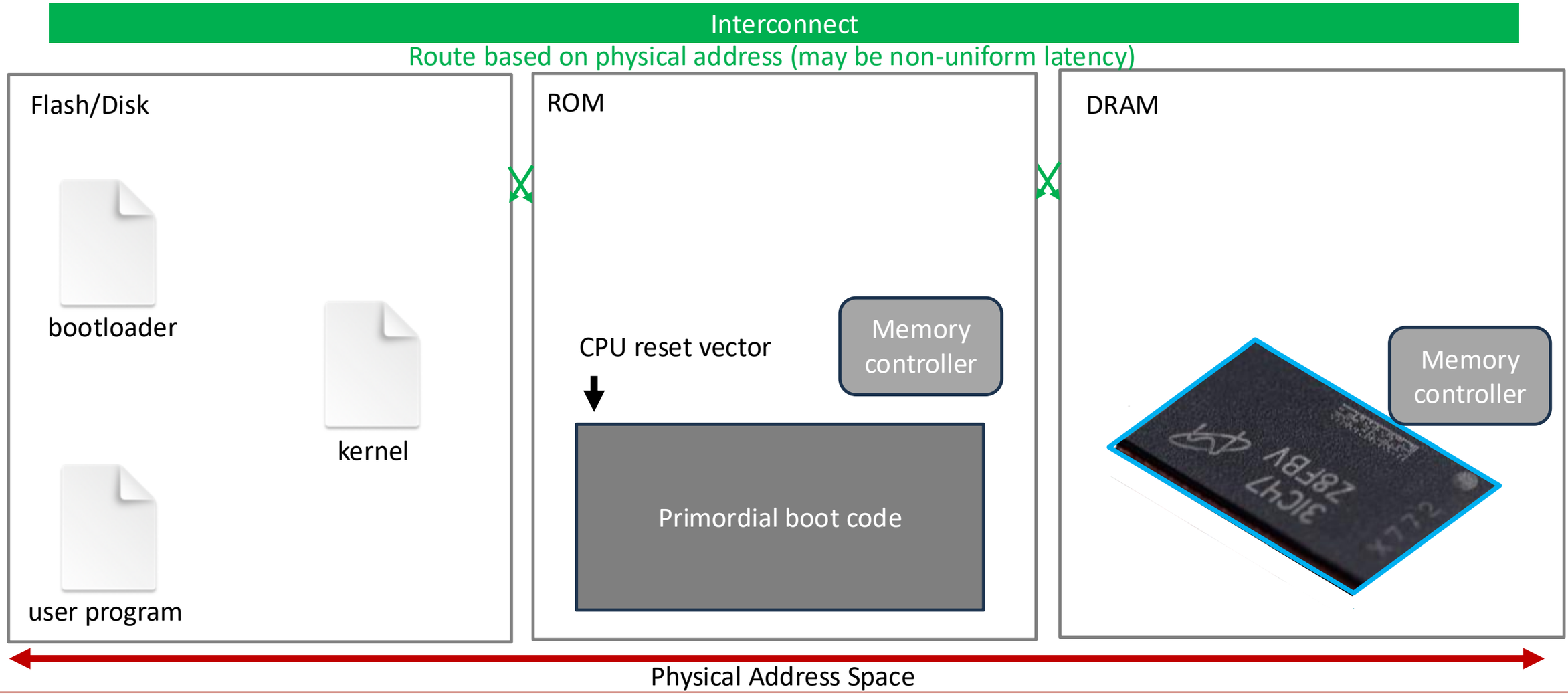
- Timesharing originally to keep machine operating at maximum bandwidth
- We've seen this before…
  - Hiding latency with independent instructions (ILP)
  - Hiding latency with independent memory accesses (MLP)
  - Hiding latency with instructions from other hardware threads (TLP on GPUs)
  - … Concept is the same
- Recall the Memory Gap
  - Extrapolate how much worse I/O is
  - This should seem like a very good idea
  - So good, in fact, that programmers might want to do this within programs (pthreads)…

Program Loading



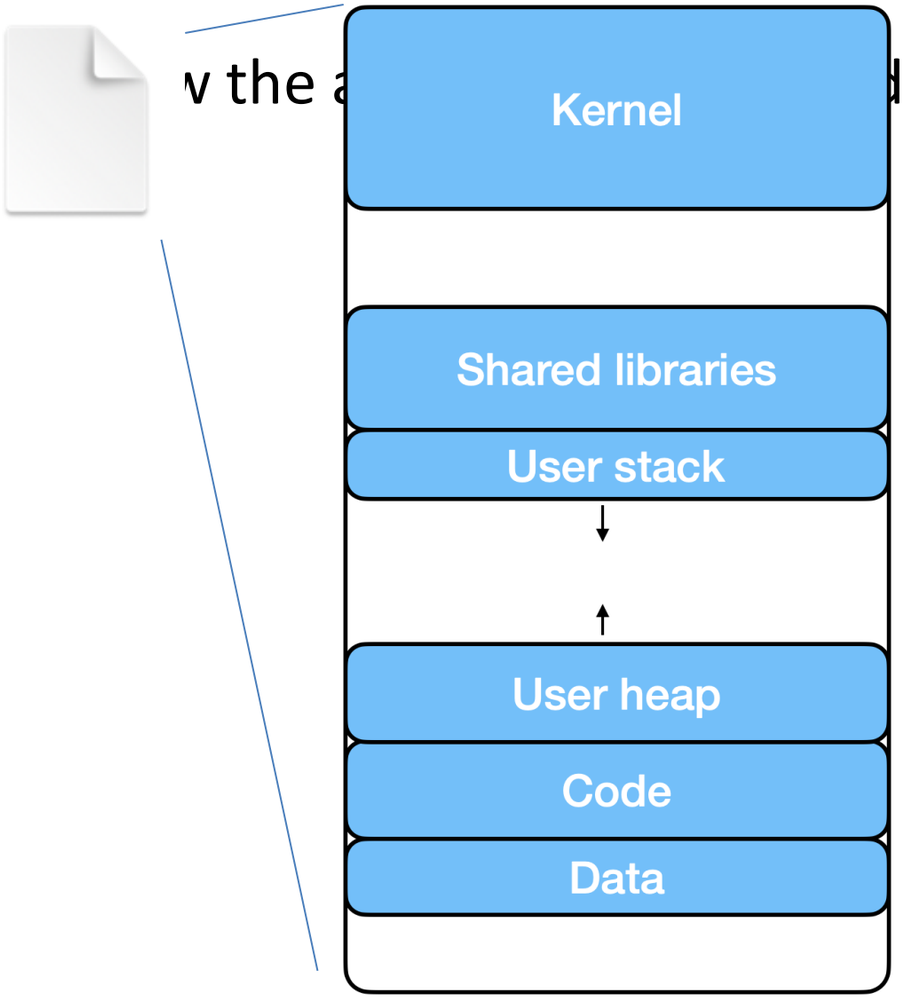
Electrical & Computer ENGINEERING

Read executable file from disk (after several layers of bootstrapping)

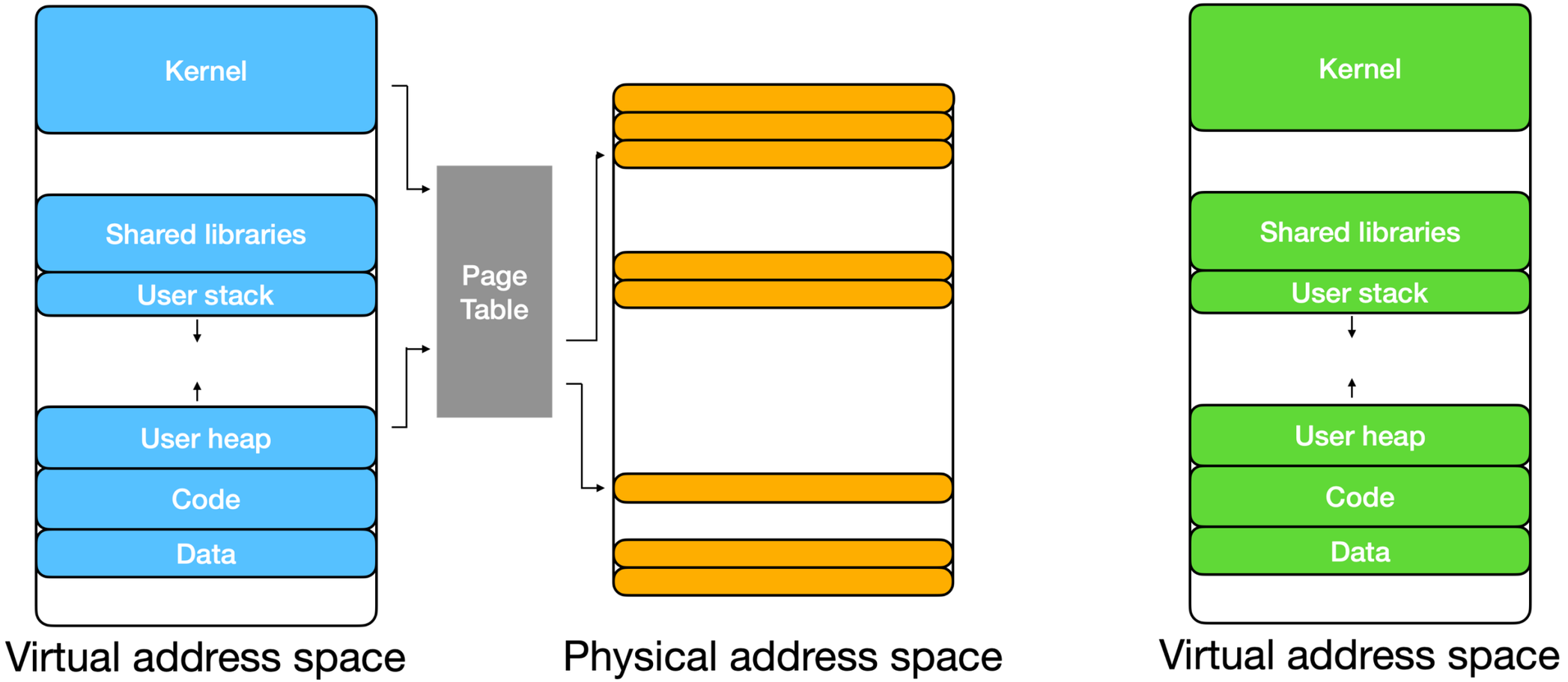


User program

- **Executable file** essentially just a specification of how the a d look
  - Virtual addresses, size, data in each section
  - Program entry point
- To load, OS needs to
  - Set up the virtual address space specified
    - Allocate physical pages
  - Set up kernel-side structures
    - I.e. entry in the thread run queue
    - Direct-map kernel memory
    - Set up and store page tables in Kernel heap

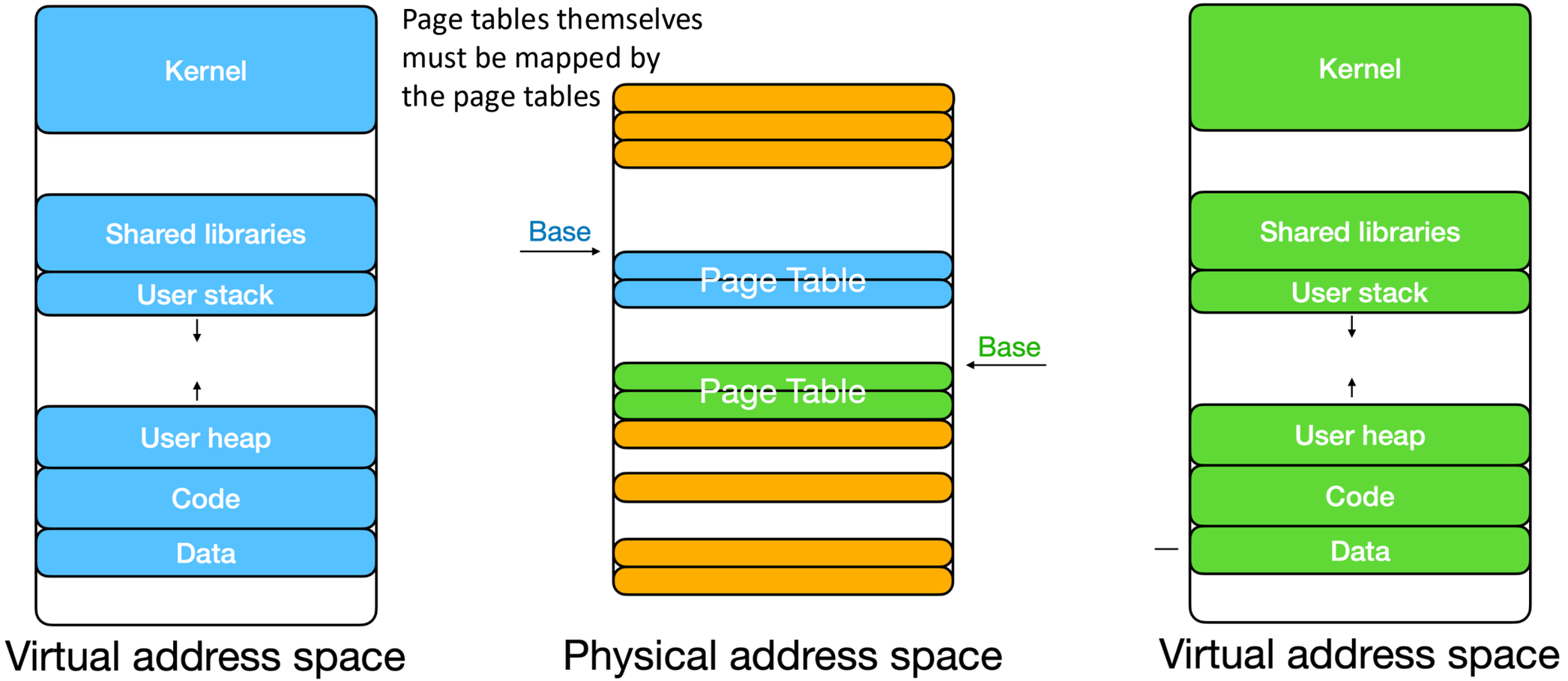


Multiple programs loaded into memory



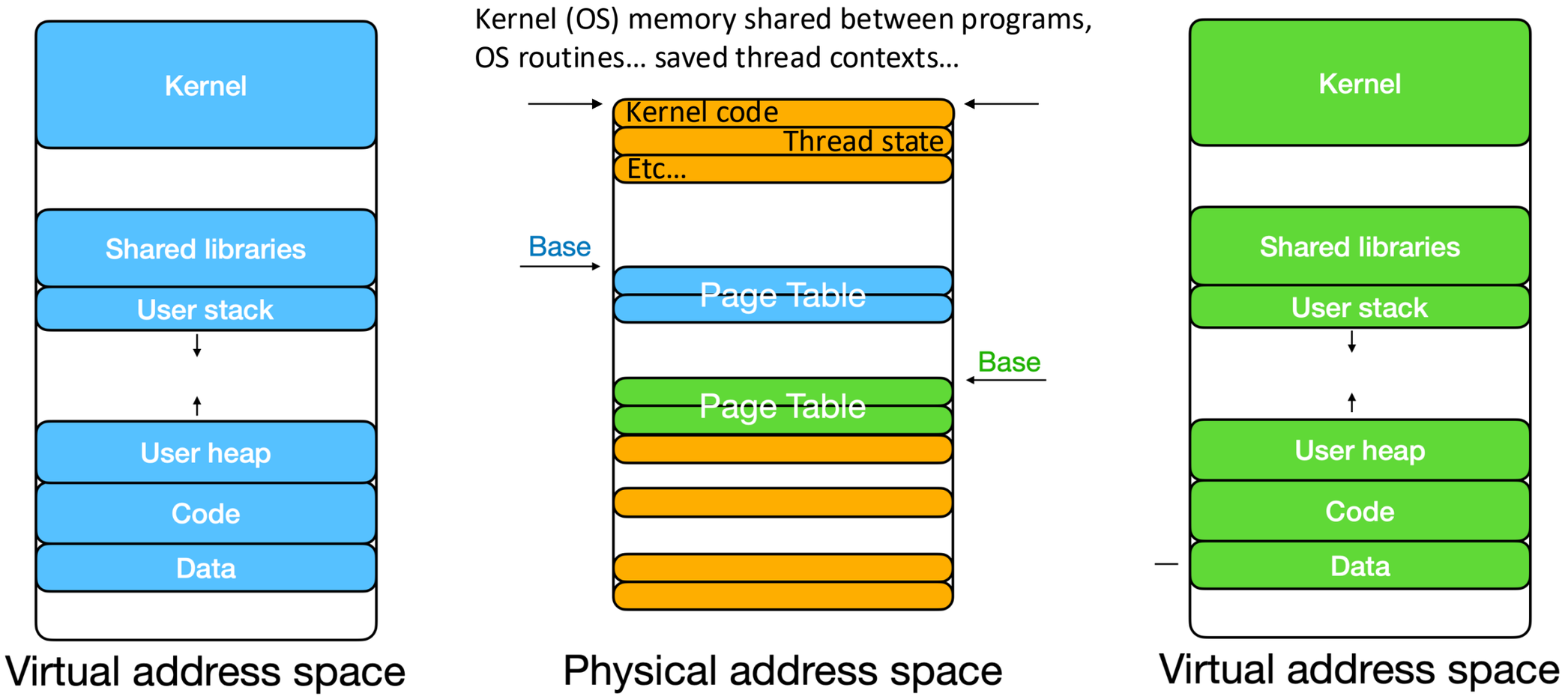
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Multiple programs loaded into memory



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Multiple programs loaded into memory

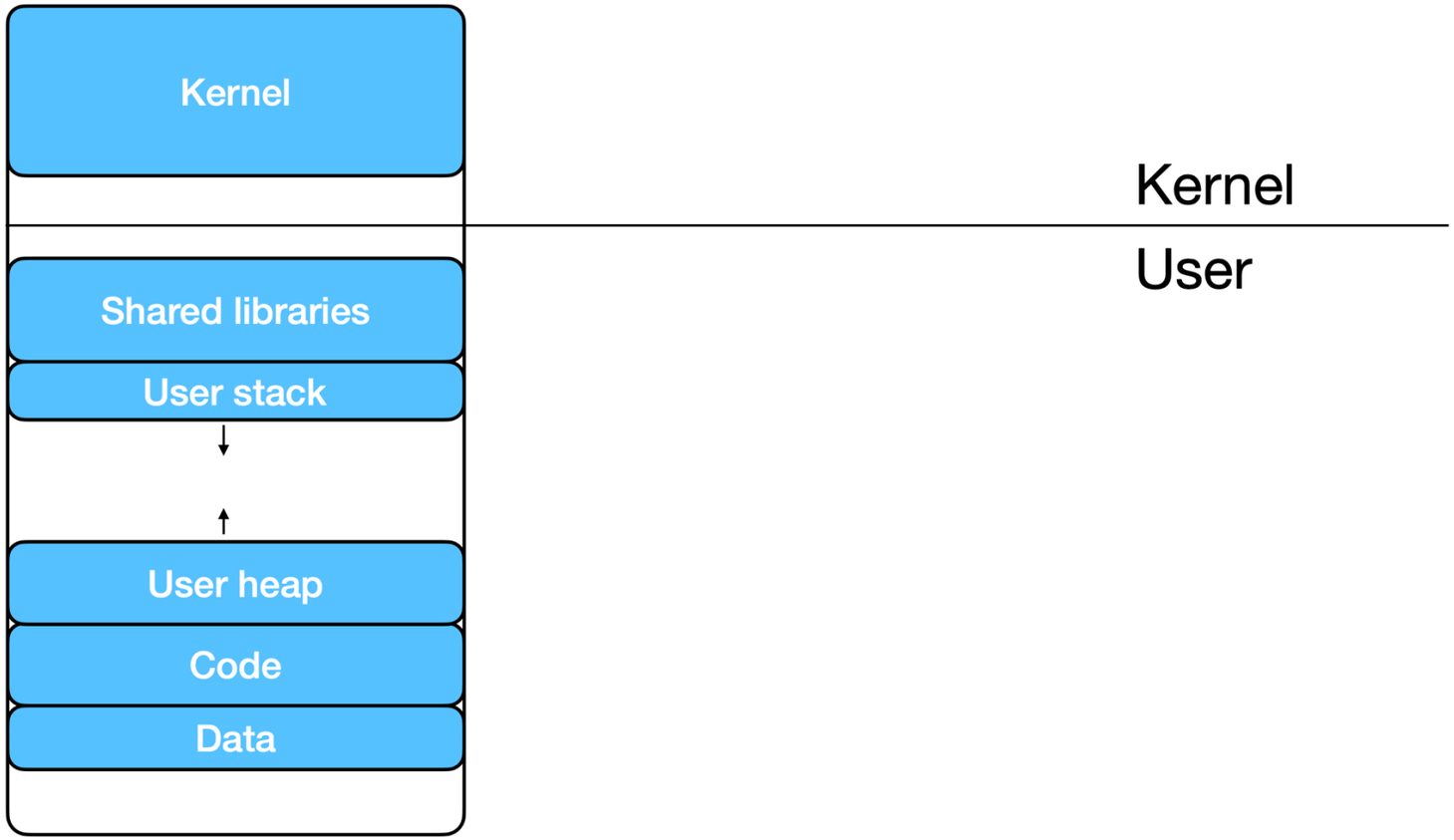


Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Hardware support for Userspace

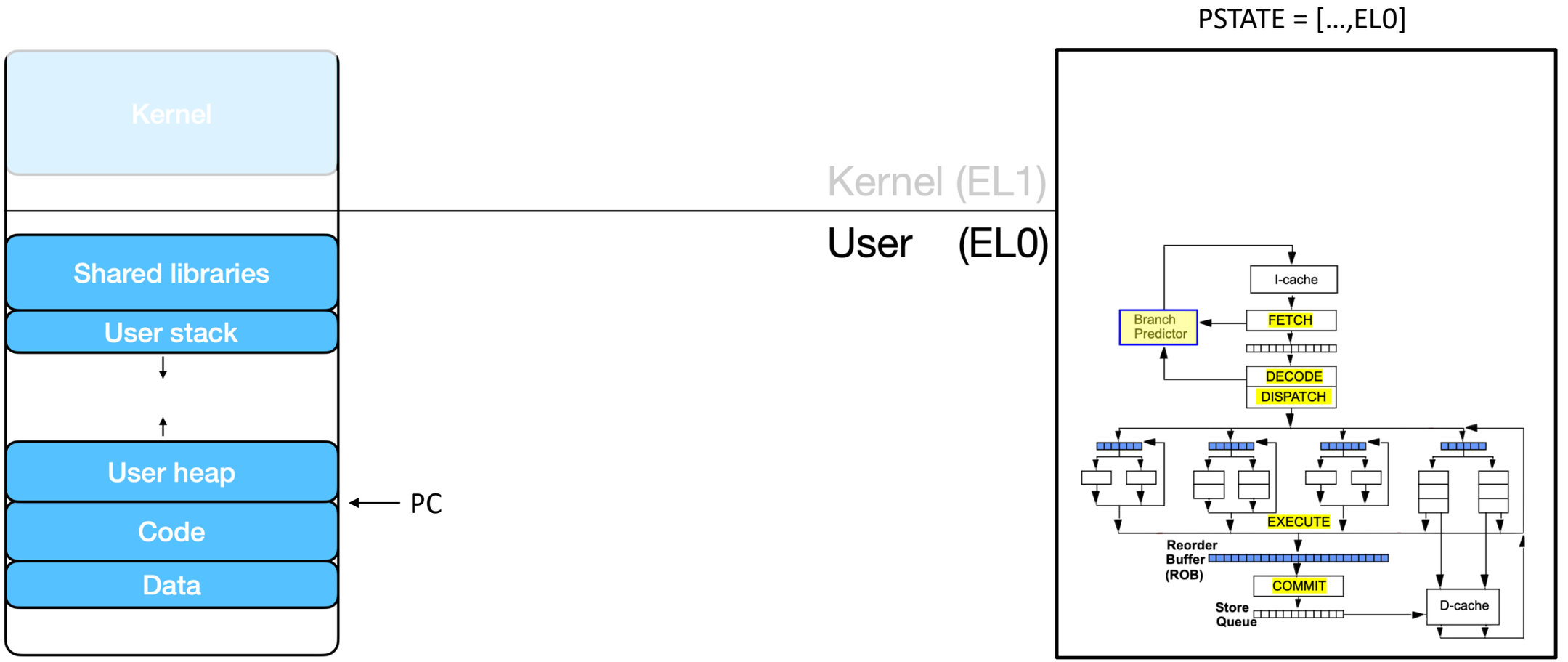


Privilege levels



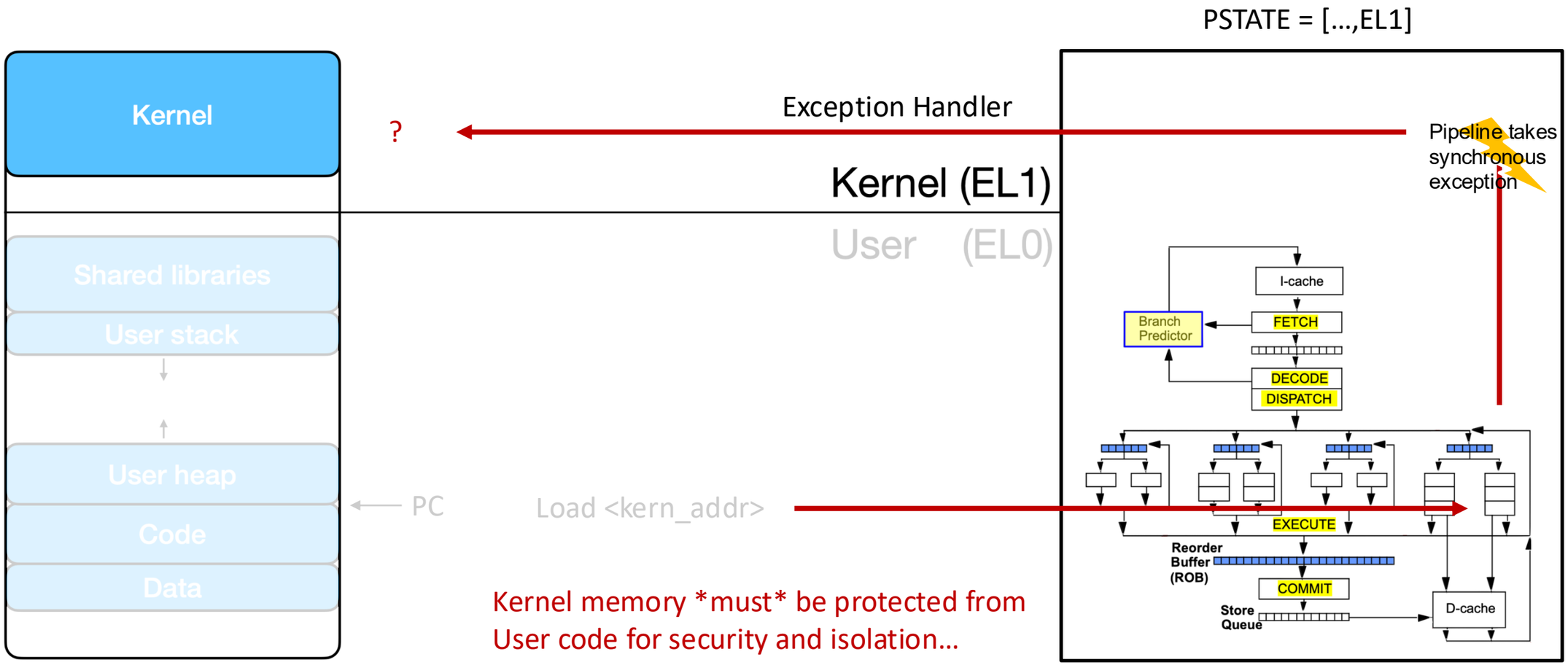
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Privilege levels



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Privilege levels

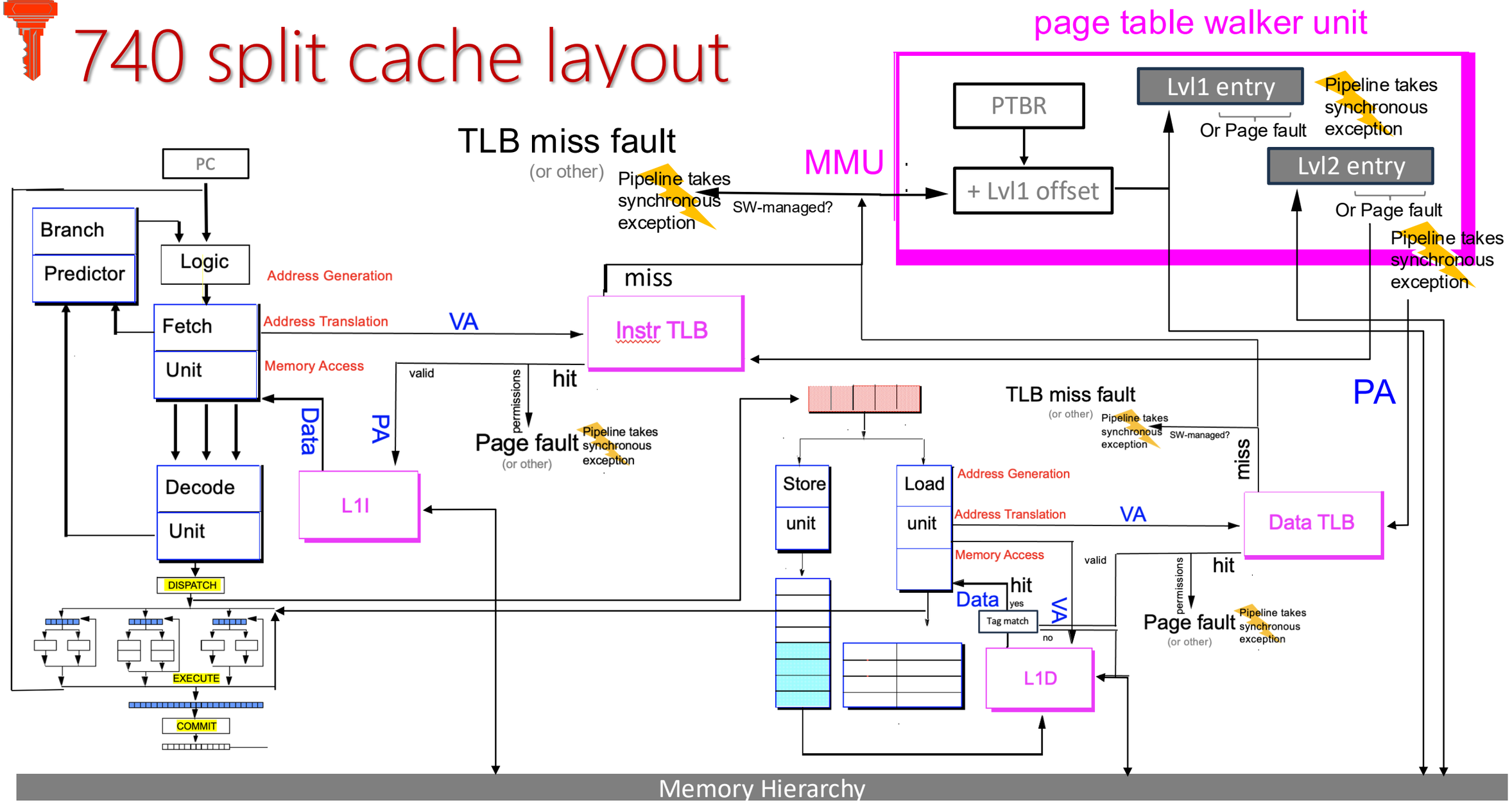


Kernel memory *must* be protected from User code for security and isolation…

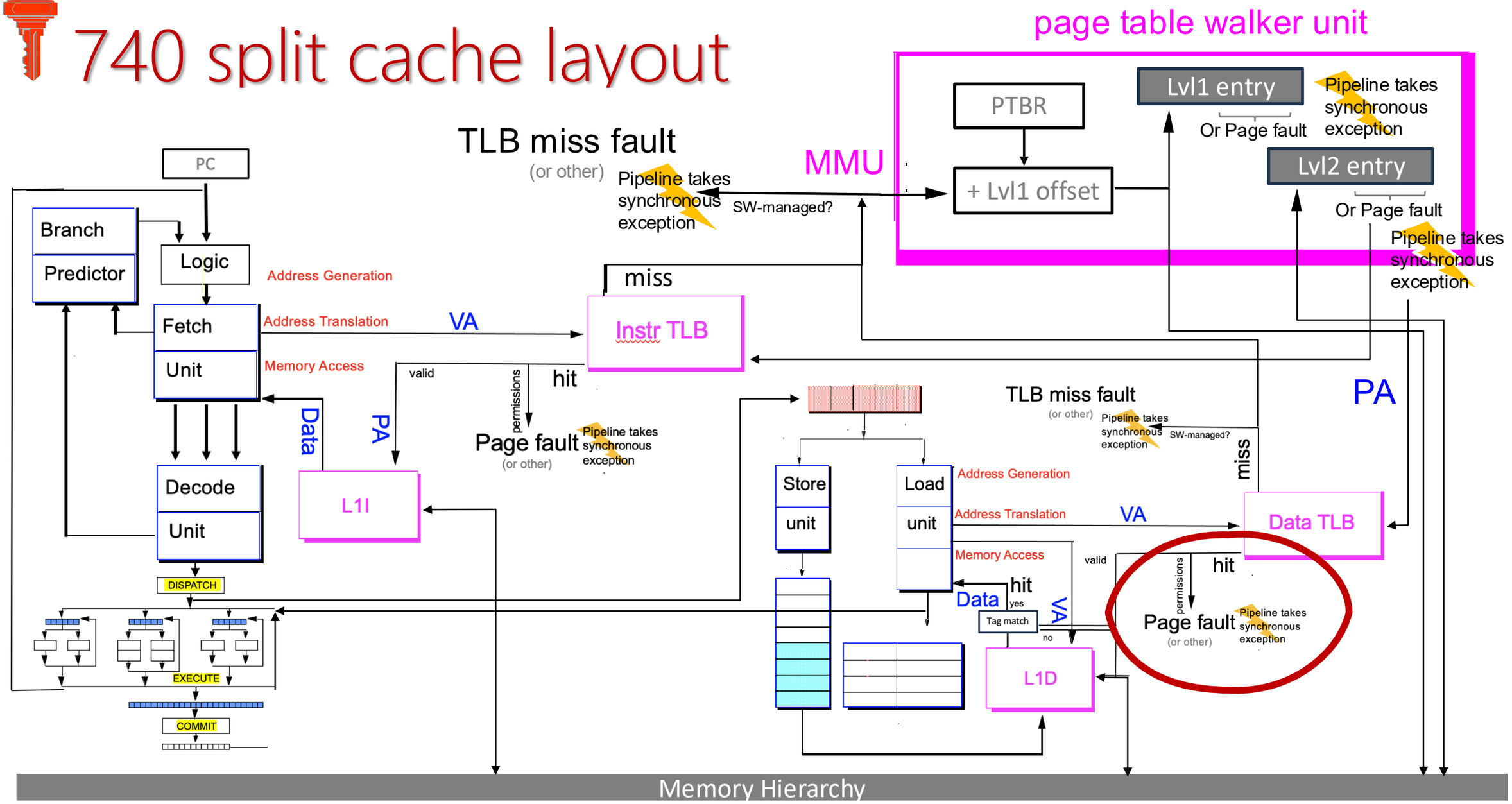
(on modern systems, not universally)

Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

740 split cache layout



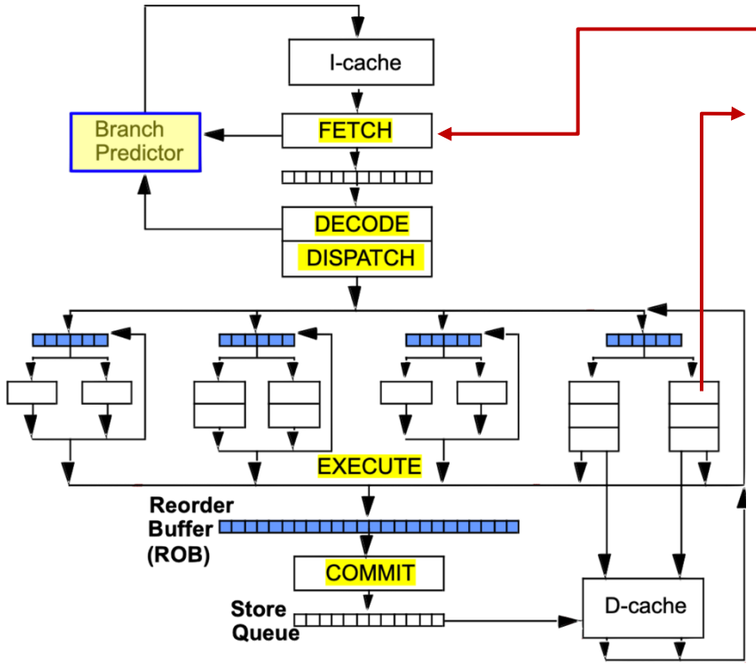
740 split cache layout



Completing the exceptional flow

Pipeline takes synchronous exception

Atomically change EL and start fetch from exception handler



Mark uOP in ROB as having hit a synchronous exception

Complete older,
Flush younger,
Fetch from exception handler

VBAR/IDT – interrupt vector base register

Permissions fault

Div-by-0

Illegal instruction

timer

keyboard

sensors

Processor is melting

other	disable_int?	VA of exception handler
other	disable_int?	VA of exception handler
...		
other	disable_int?	VA of exception handler

Picture this as another structure the OS is responsible for managing.

Like page table, must walk (in hardware or software) on an exception

...

Exception/Interrupt Vector Table

Memory Hierarchy

Handling an exception

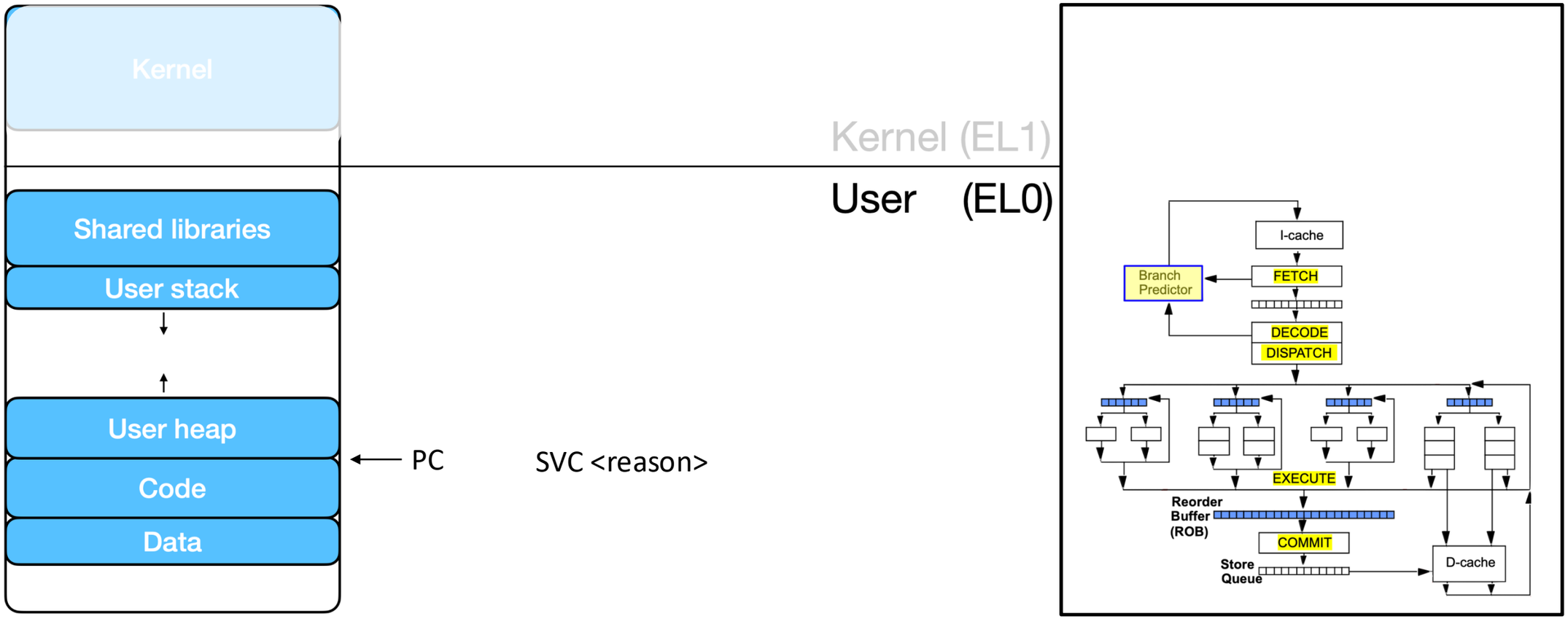
- Largely up to the Operating System
  - Default may be to simply kill the thread that hit the exception
  - Also consider letting the application handle the exception
    - return not to the faulting PC but to a function pointer the application has registered with the OS
    - Recall signal handlers from 18/15-213/513/613.
  - Linux sends a SIGSEGV signal (i.e. SEGFAULT)
- Some exceptions may not be the program's fault
  - "Processor is Melting" exception will not be fixed by killing random threads
  - Device interrupts almost certainly have nothing to do with the program running

Handling an exception

- What if no handler is installed?
  - “No-handler” handler
    - What if no handler for “No-handler”?
      - “No-No-handler” handler
        - Shall I go on?
- What if hit an exception in an exception handler?
  - Double fault handler
    - What if hit an exception in a Double fault handler?
      - Triple fault handler
        - Shall I go on?
- Everything that can go wrong will go wrong
  - And a lot of complexity is pawned off to the OS

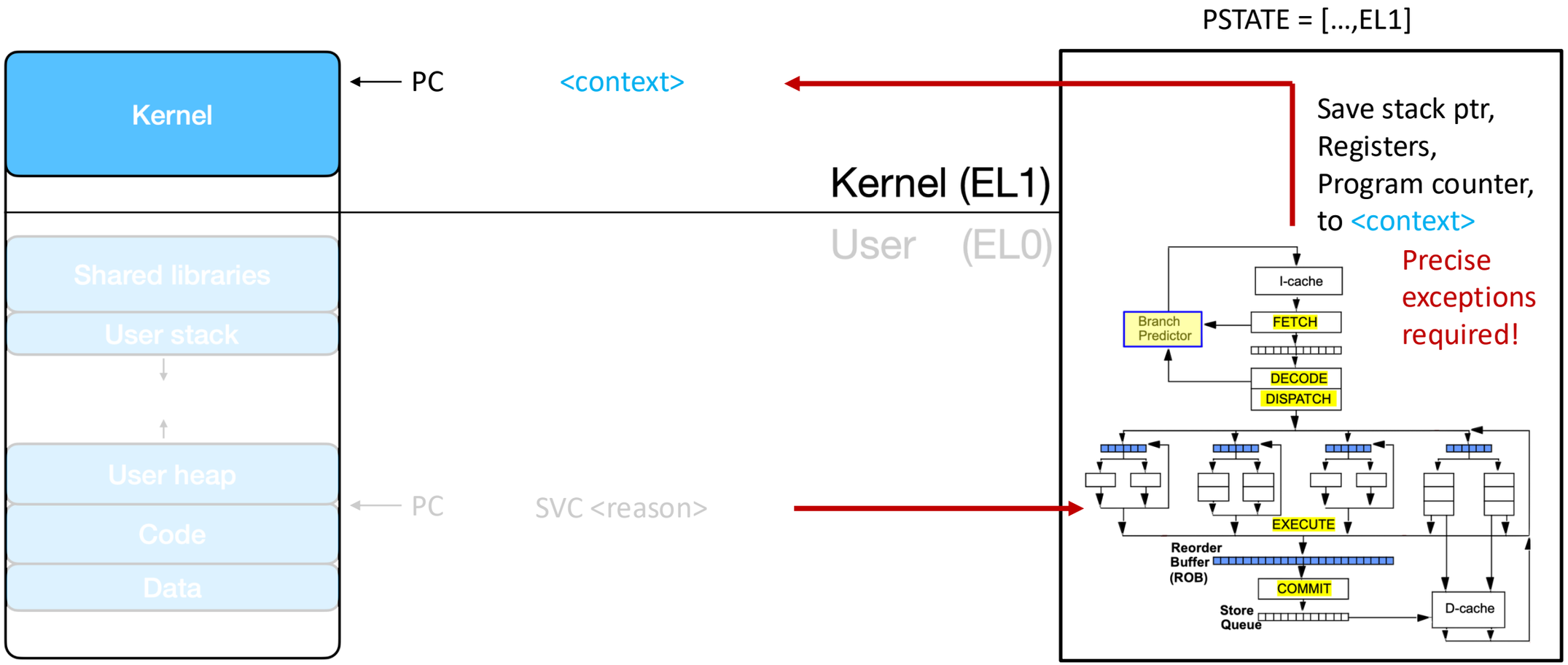
Voluntary Exception (synchronous **trap**)

PSTATE = […,EL0]



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

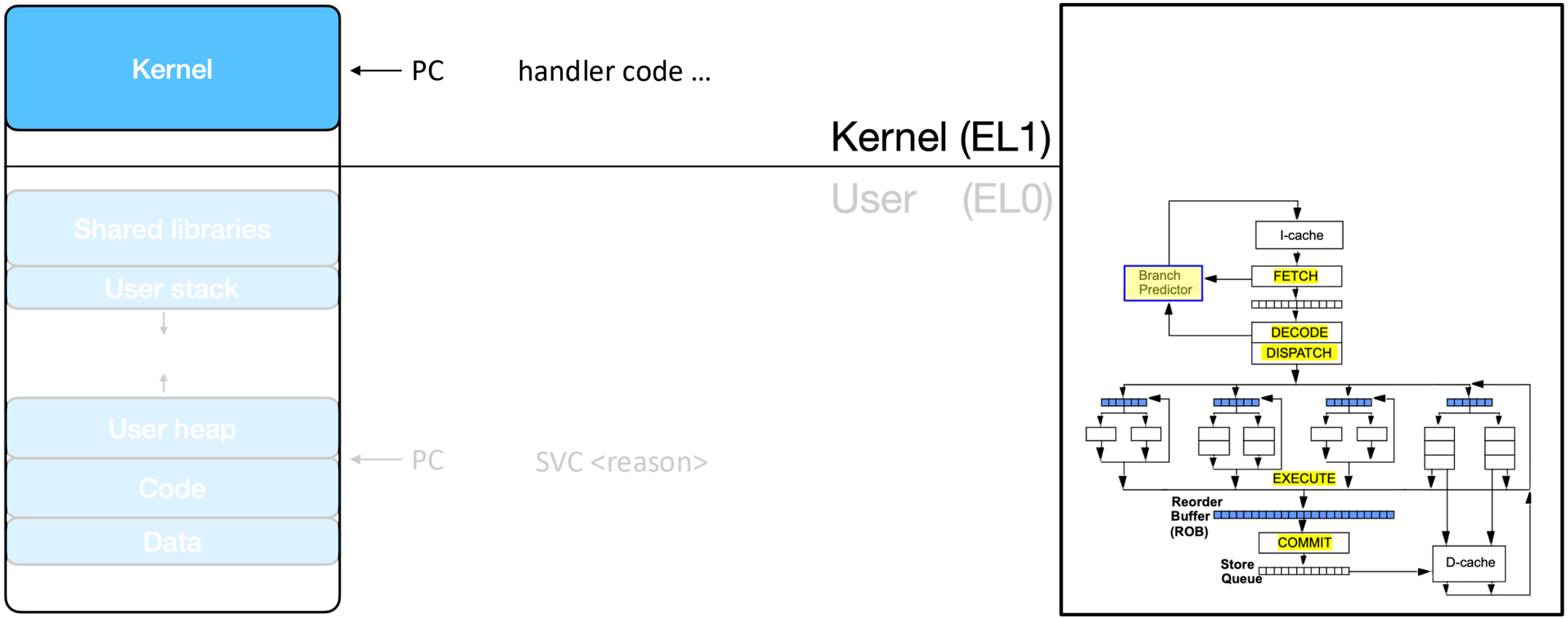
Voluntary Exception (synchronous **trap**)



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

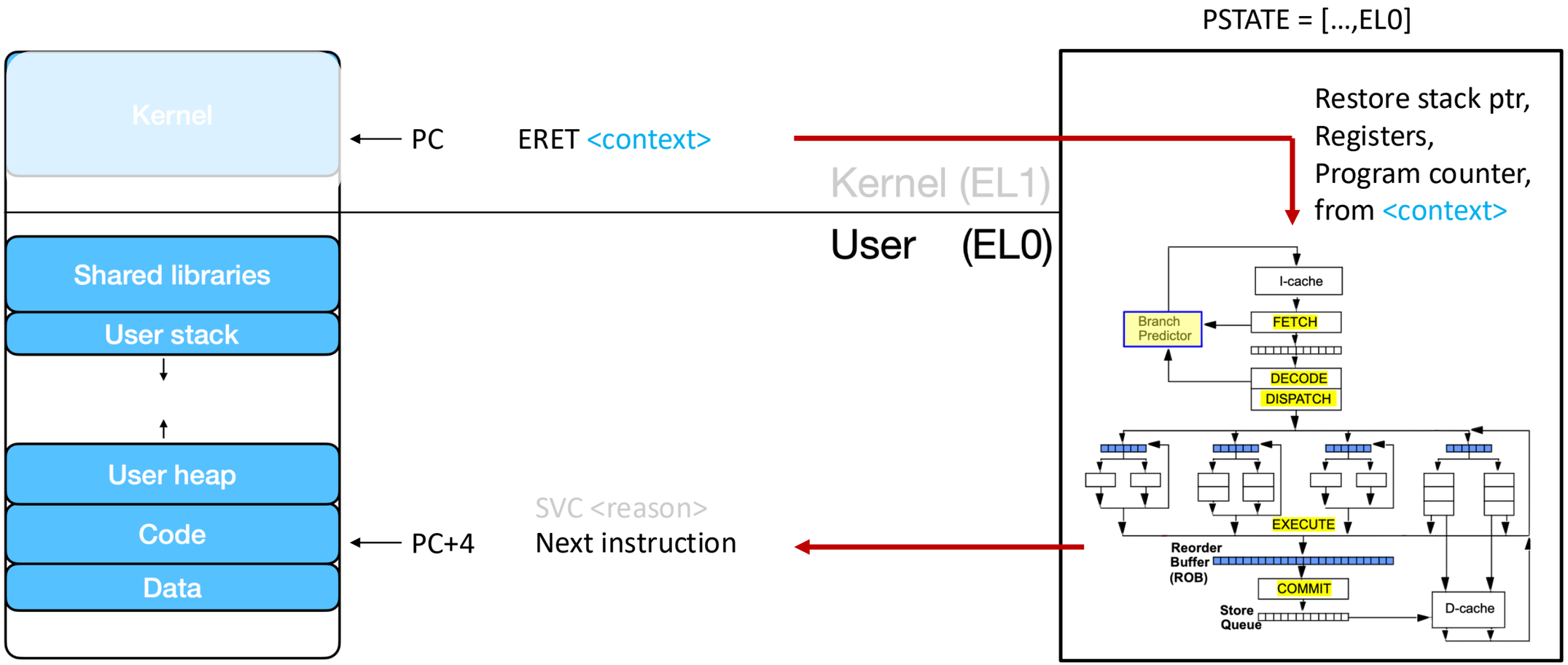
Voluntary Exception (synchronous **trap**)

PSTATE = […,EL1]



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Voluntary Exception (synchronous **trap**)

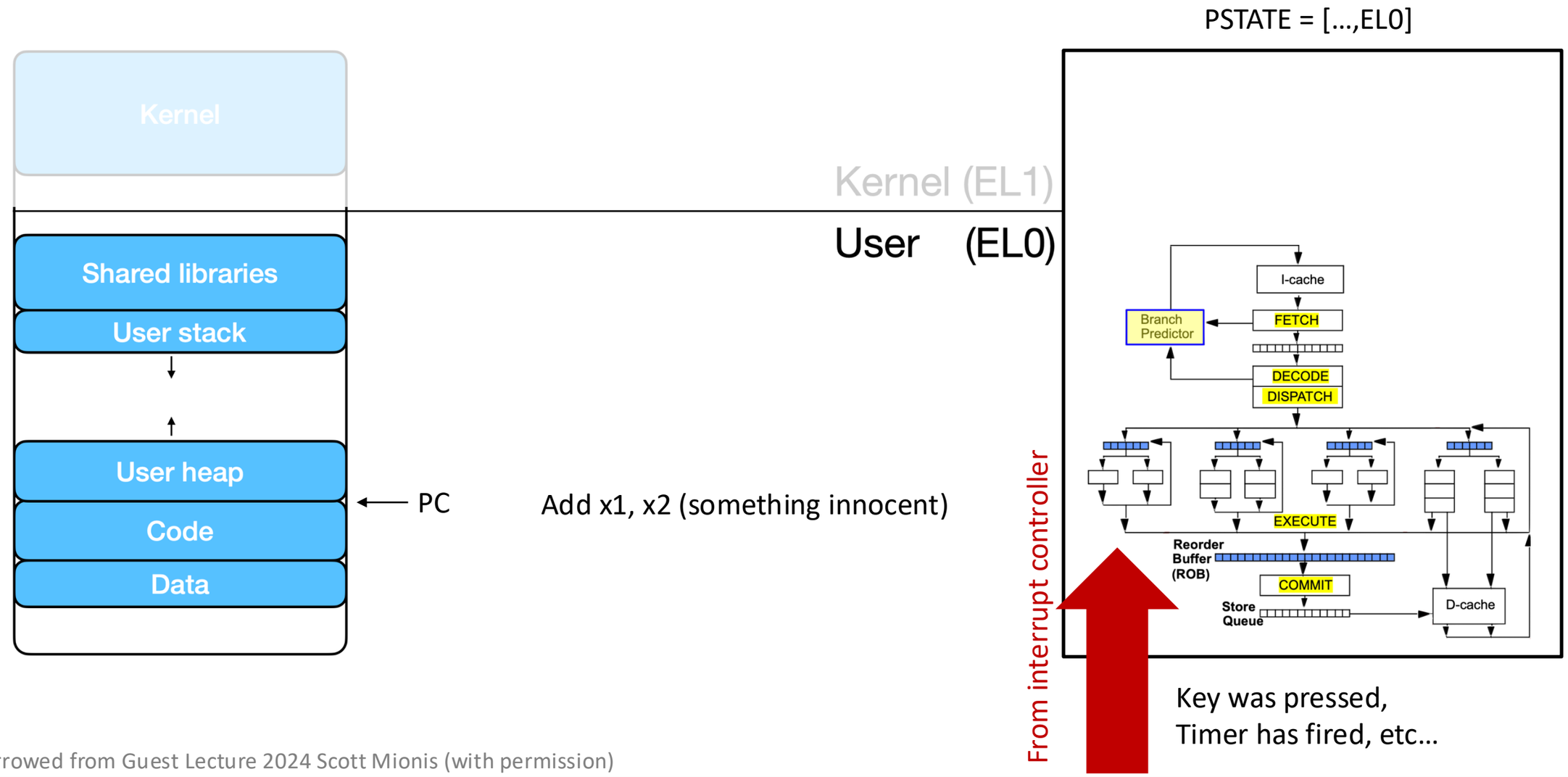


Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

What are System Calls used for?

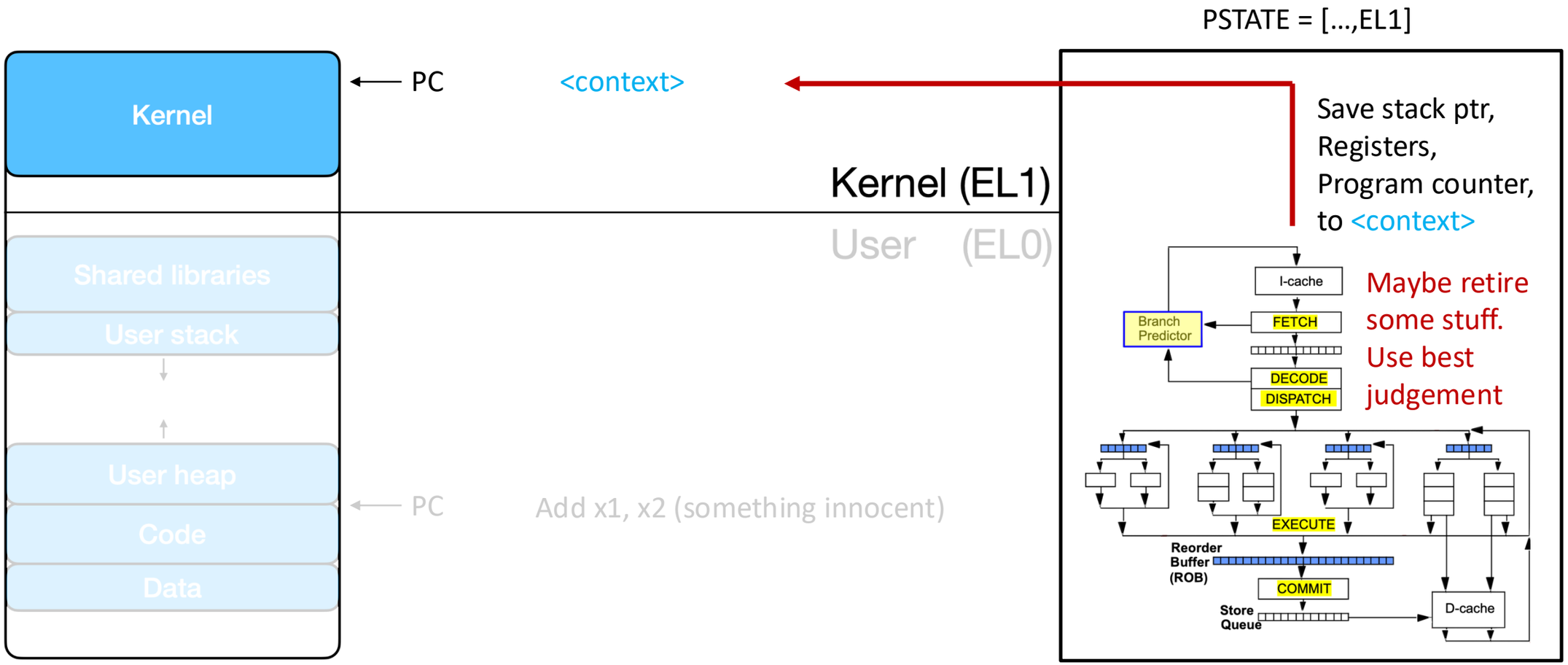
- Process create (fork, clone, posix_spawn, exec)
- Thread create (thr_fork, clone)
- I/O operations (read disk sector, move file)
- Memory allocation/deallocation
  - Request new physical page allocated at VA X (stack growth, heap extend)
  - Free physical page allocated at VA Y
  - Memory-map a file to a specified VA range (uncacheable memory… handled by MMU)
- Other…

Involuntary Exception (async **interrupt**)



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

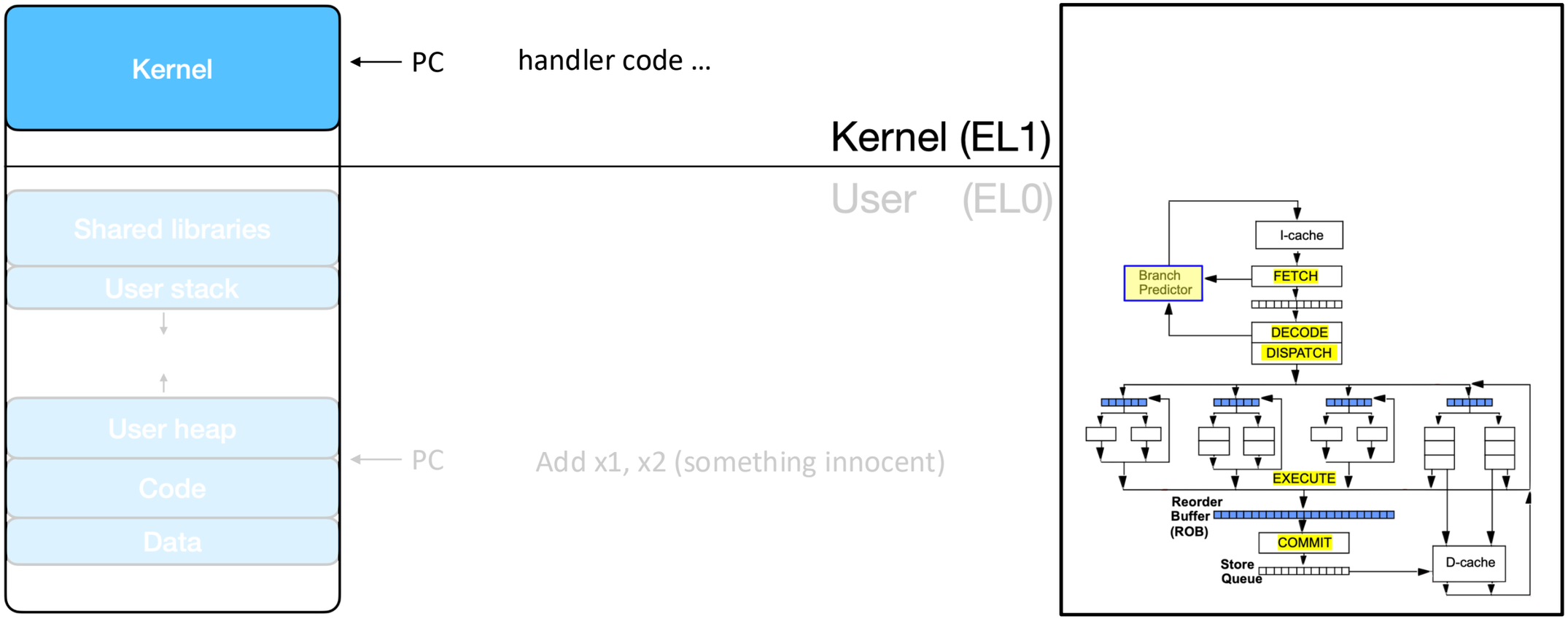
Involuntary Exception (async **interrupt**)



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

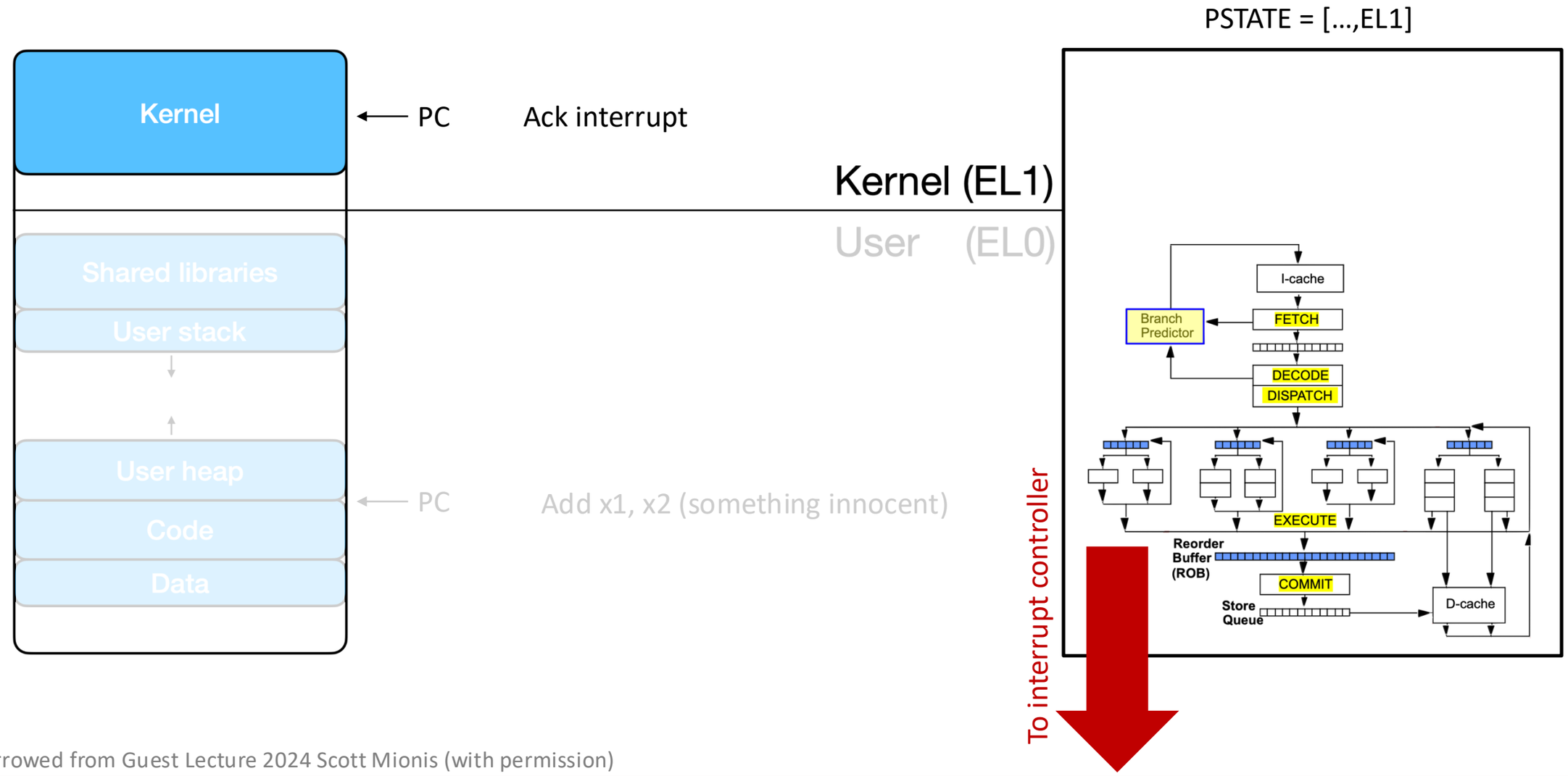
Involuntary Exception (async **interrupt**)

PSTATE = […,EL1]



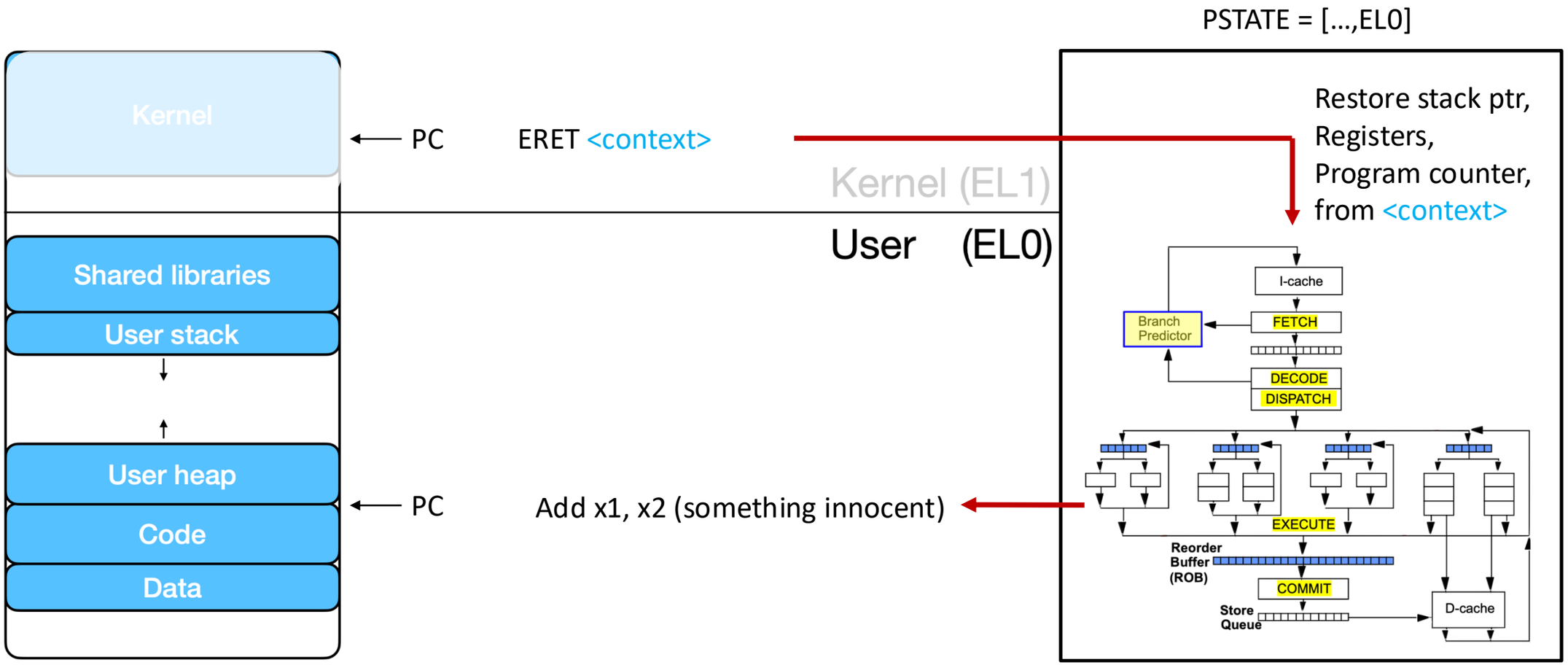
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Involuntary Exception (async **interrupt**)



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Involuntary Exception (async **interrupt**)

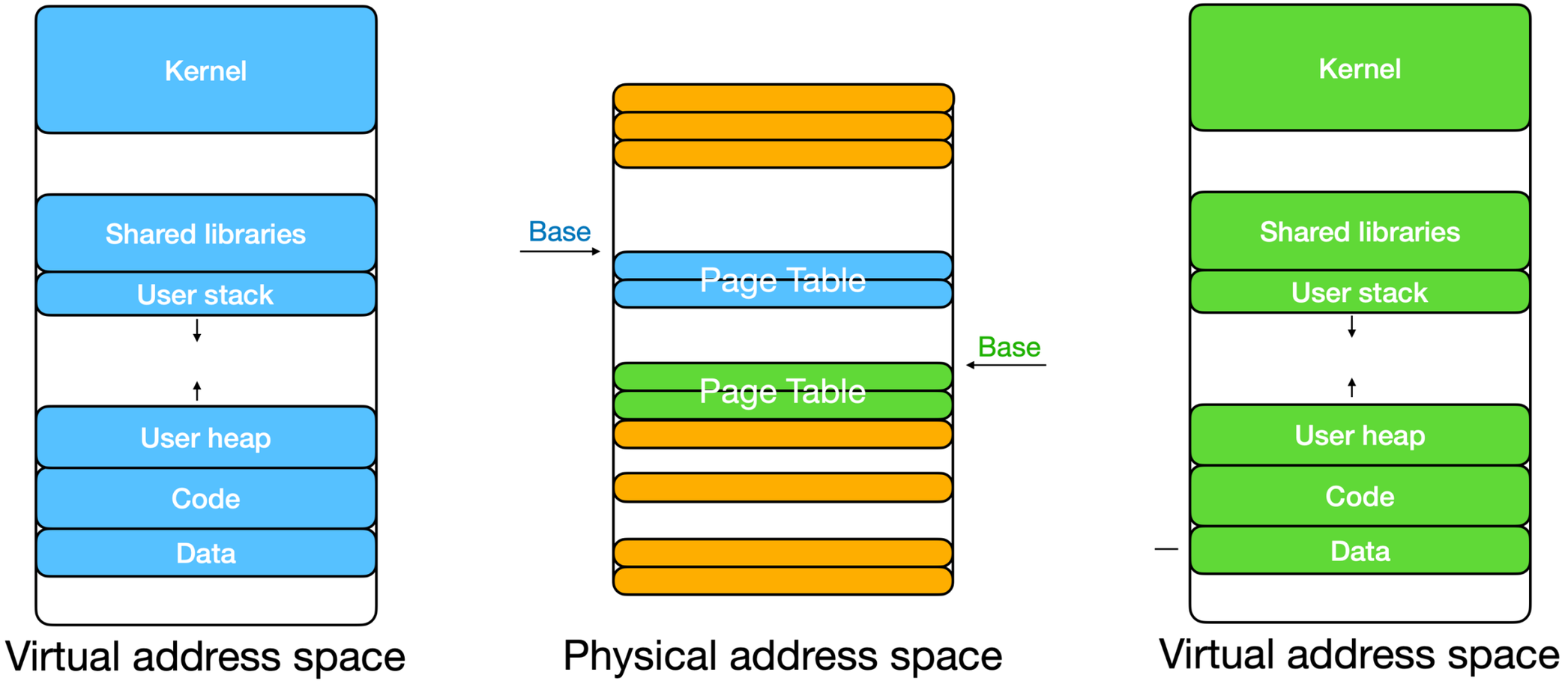


Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Handling an Interrupt

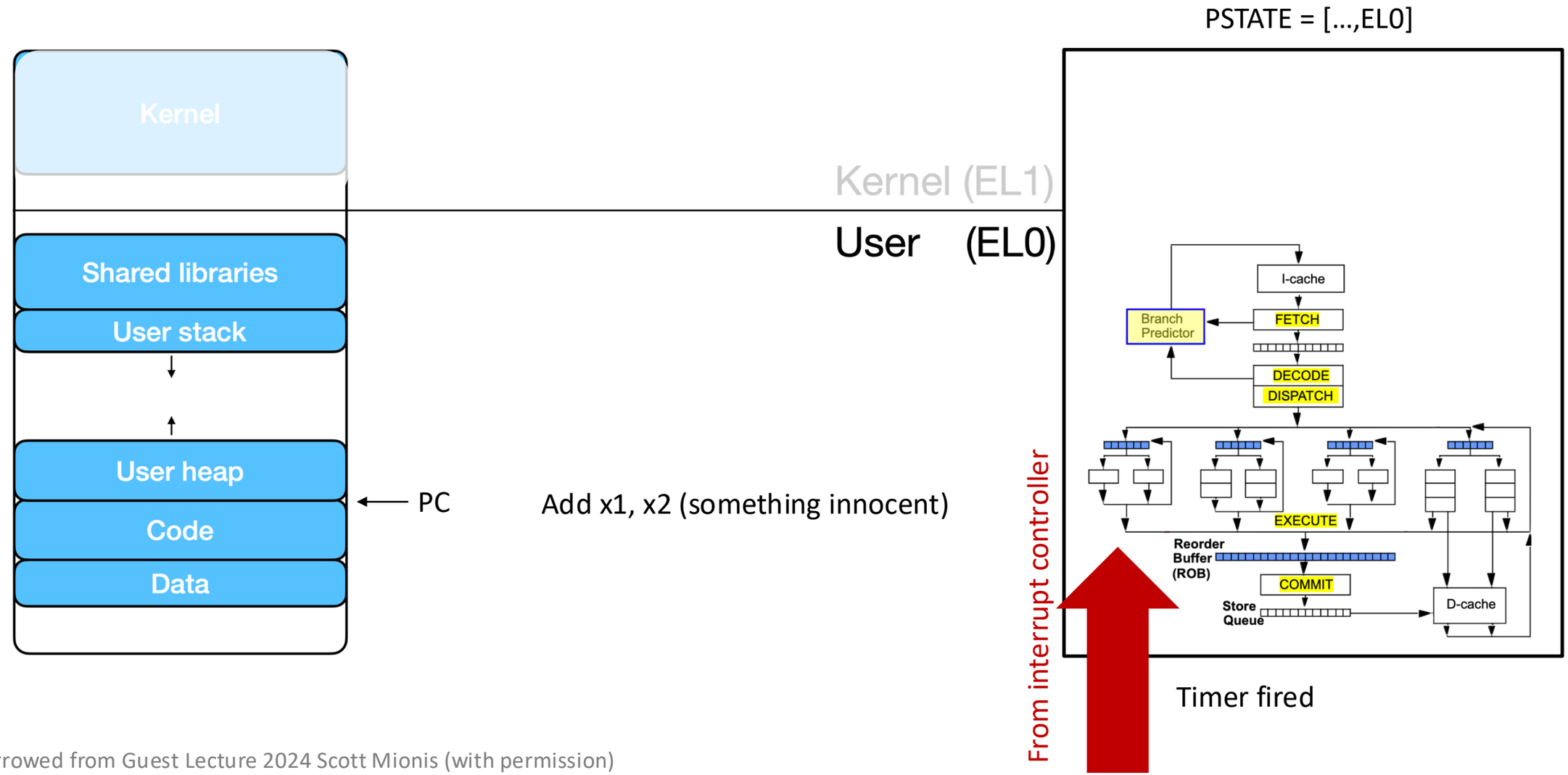
- Interrupts typically collected/injected to the CPU by the ***interrupt controller***
  - ***Example*** – Arm GIC or x86 APIC
  - Usually very complex pipelines, programmed by kernel code via system registers
  - Typically blocks future interrupts (by type or globally) until ack
    - OS still tries to make interrupt handlers re-entrant (i.e. recursive) where possible for QoS
- Processors may send each other interrupts and this may be latency-sensitive
  - ***Example –*** IPIs for message-passing utilities
- Timer interrupts often leveraged to implement timesharing
  - I.e. give each thread a time quanta, and switch at fine granularity

Multiple programs loaded into memory



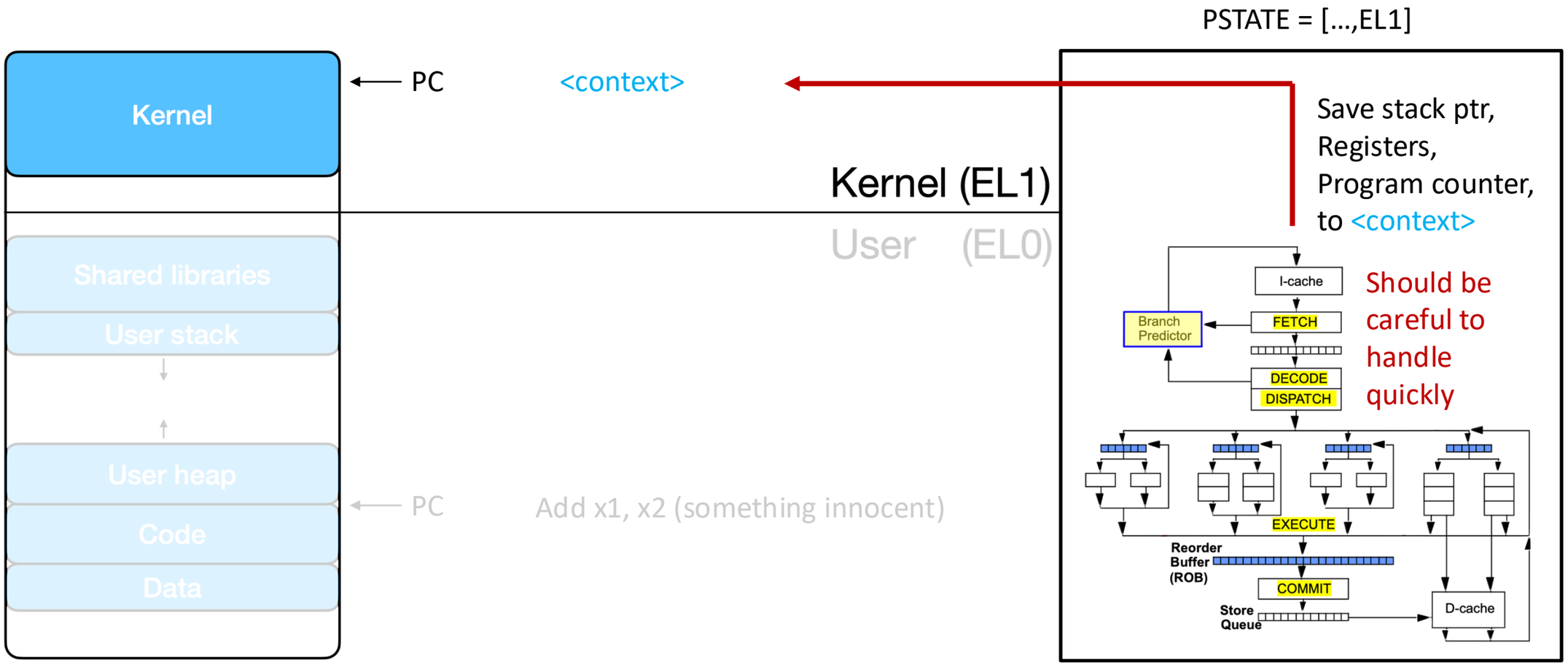
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Context switch



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

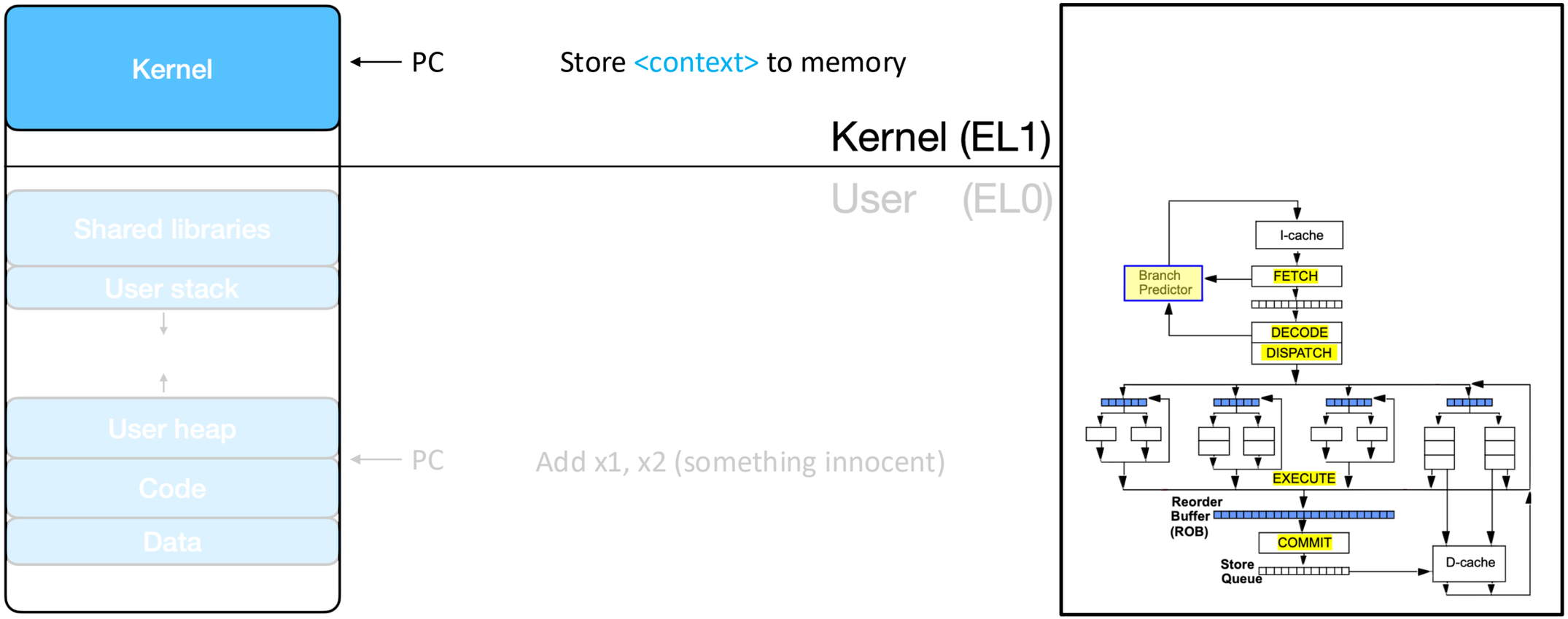
Context switch



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

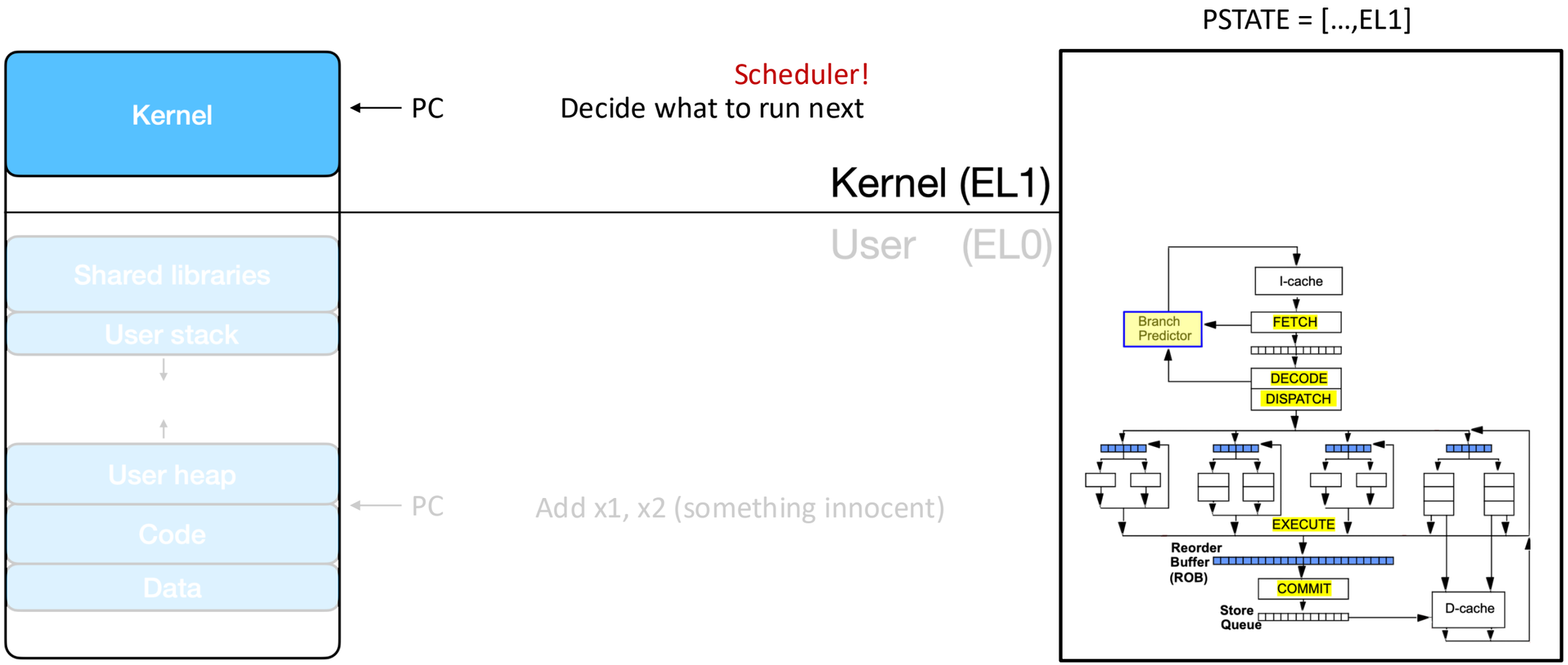
Context switch

PSTATE = […,EL1]



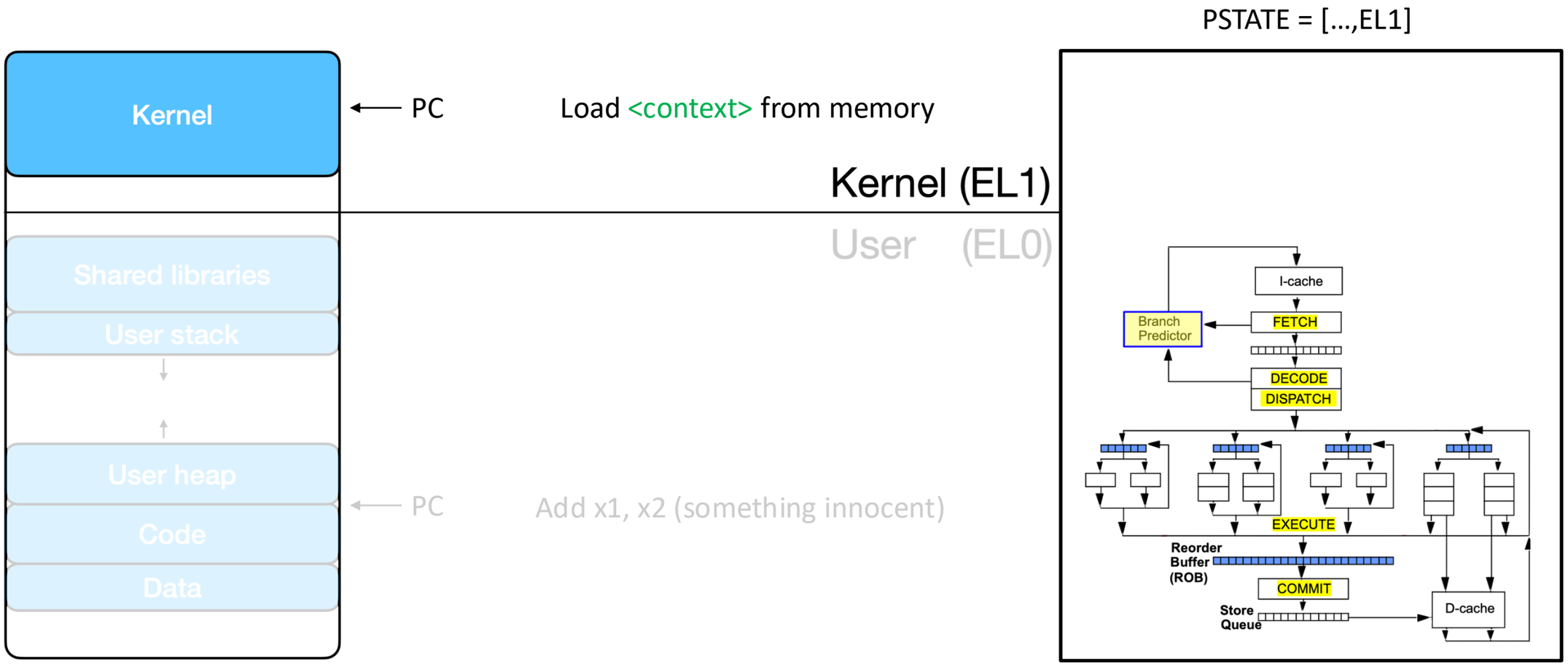
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Context switch



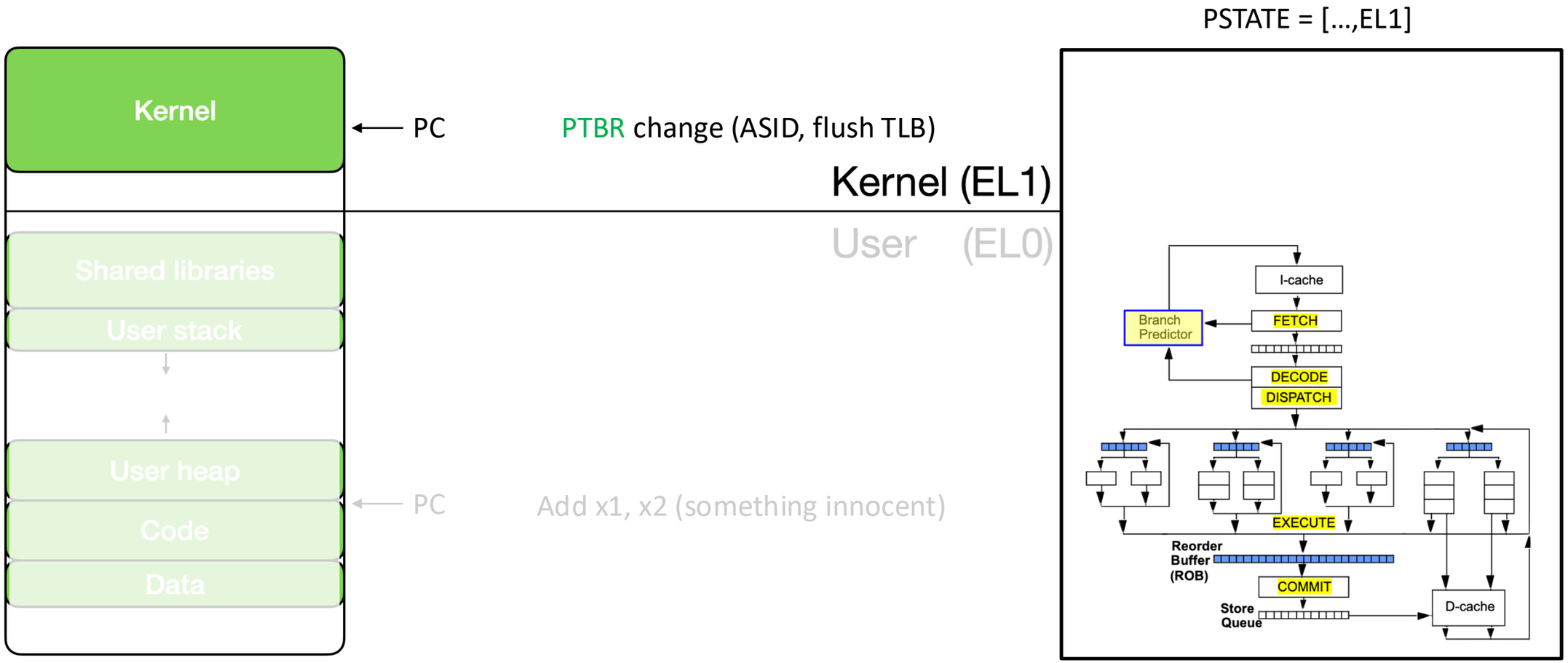
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Context switch



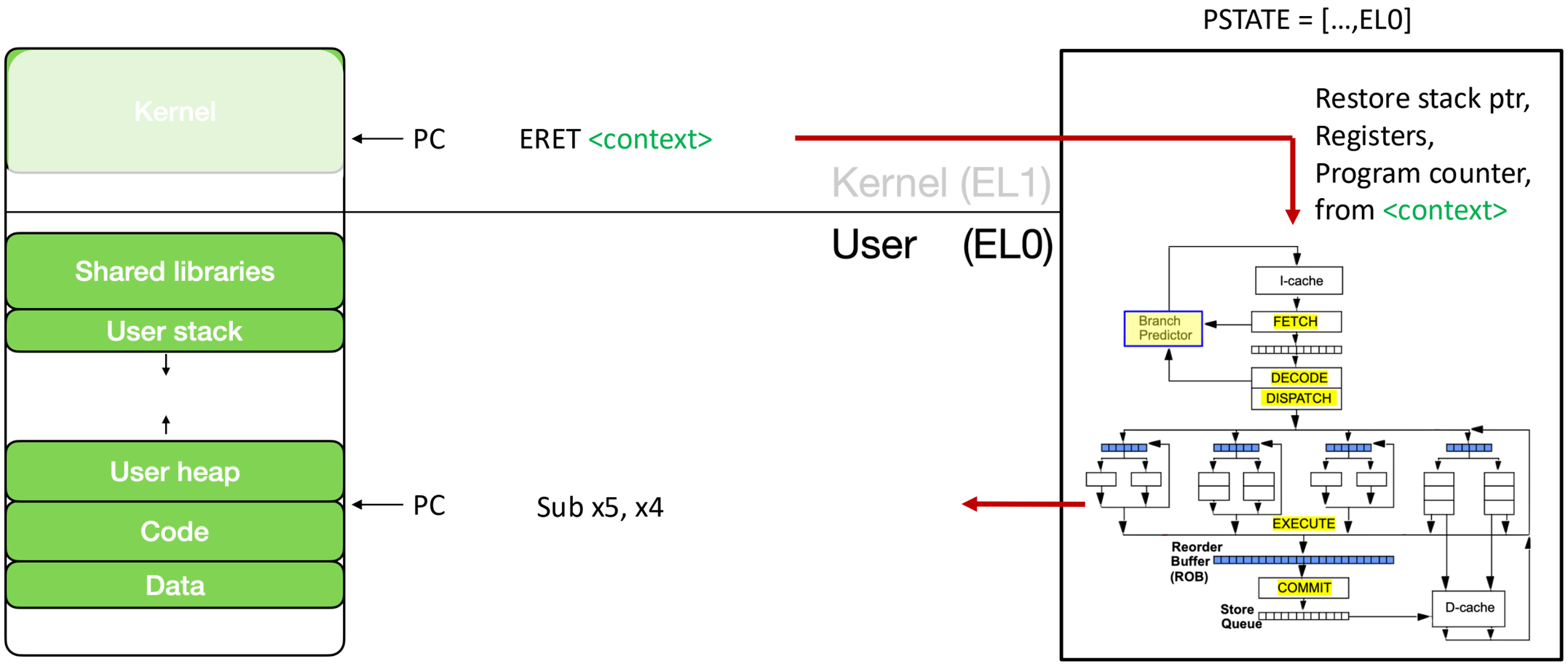
Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Context switch



Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Context switch

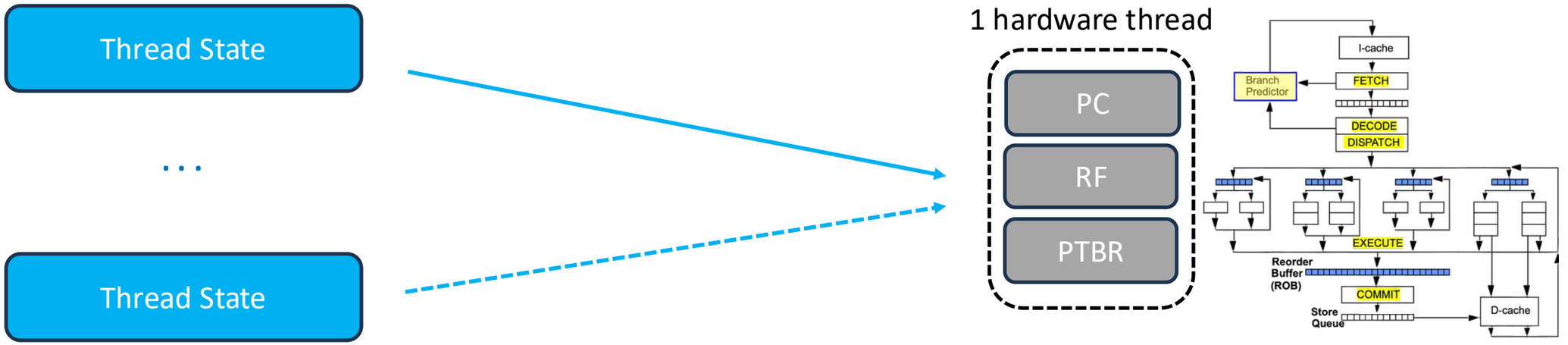


Content borrowed from Guest Lecture 2024 Scott Mionis (with permission)

Handling a Context Switch

- Similar flow to returning from an interrupt, just returning to a different thread than the thread that took the interrups
- Scheduler can be ***extremely complex***
  - Context Switch overhead?

M program threads
(organized by OS into a queue in memory)



Handling a Context Switch

- Similar flow to returning from an interrupt, just returning to a different thread than the thread that took the interrups
- Scheduler can be ***extremely complex***
  - Context Switch overhead?
    - *VIVT cache flush*
    - *Potentially TLB flush/barriers* *(recall software coherence for split TLBs)*
    - *1000's to 10,000's of instructions*
    - *Almost certainly OS pages (code/data) are not in cache*
    - *Almost certainly branch predictor state is not applicable to new thread*
    - *Almost certainly all other predictor state is not applicable to new thread*
    - *Almost certainly new thread's working set is not in the cache*

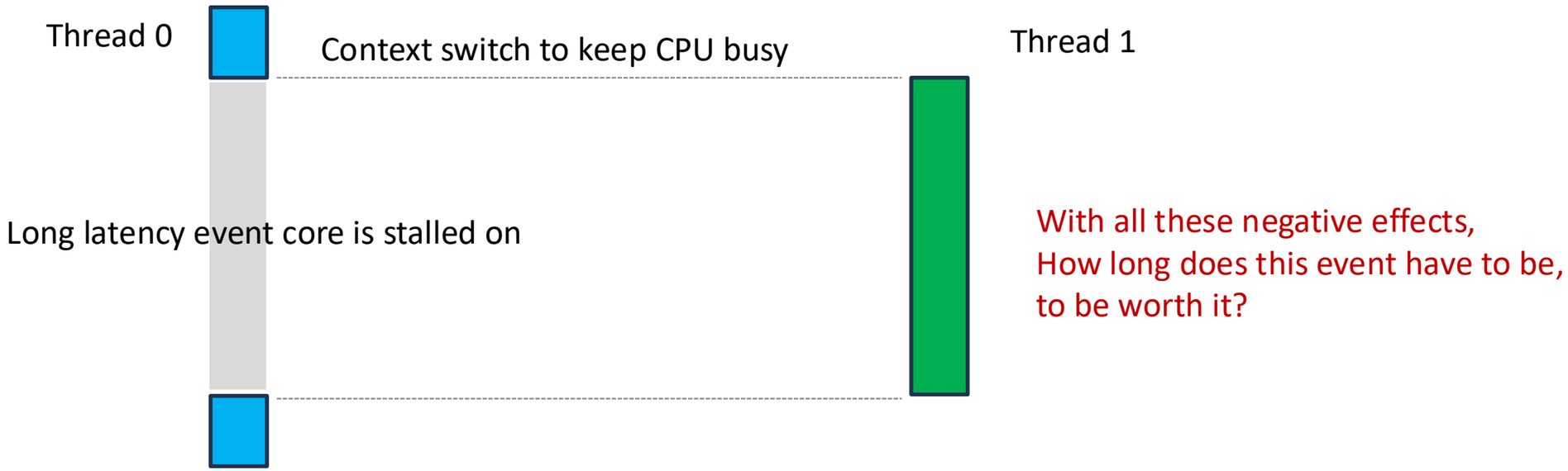
Handling a Context Switch

- Similar flow to returning from an interrupt, just returning to a different thread than the thread that took the interrups
- Scheduler can be ***extremely complex***
  - Context Switch overhead?
    - *VIVT cache flush*
    - *Potentially TLB flush/barriers* *(recall software coherence for split TLBs)*
    - *1000's to 10,000's of instructions*
    - *Almost certainly OS pages (code/data) are not in cache*
    - *Almost certainly branch predictor state is not applicable to new thread*
    - *Almost certainly all other predictor state is not applicable to new thread*
    - *Almost certainly new thread's working set is not in the cache*
    - *If old thread is run on a different core later on it almost certainly needs to snoop this one for data*

These effects all get worse with cache hierarchy complexity, predictor complexity…

The "Context Switch Gap" (not a real term I made this up)

- Modern CPUs rely heavily on ***warmup***
  - Larger caches are necessary but only kick in when working set is loaded
  - Larger branch predictors only kick in when sufficiently trained
- Software context switching is fundamentally a latency-hiding technique

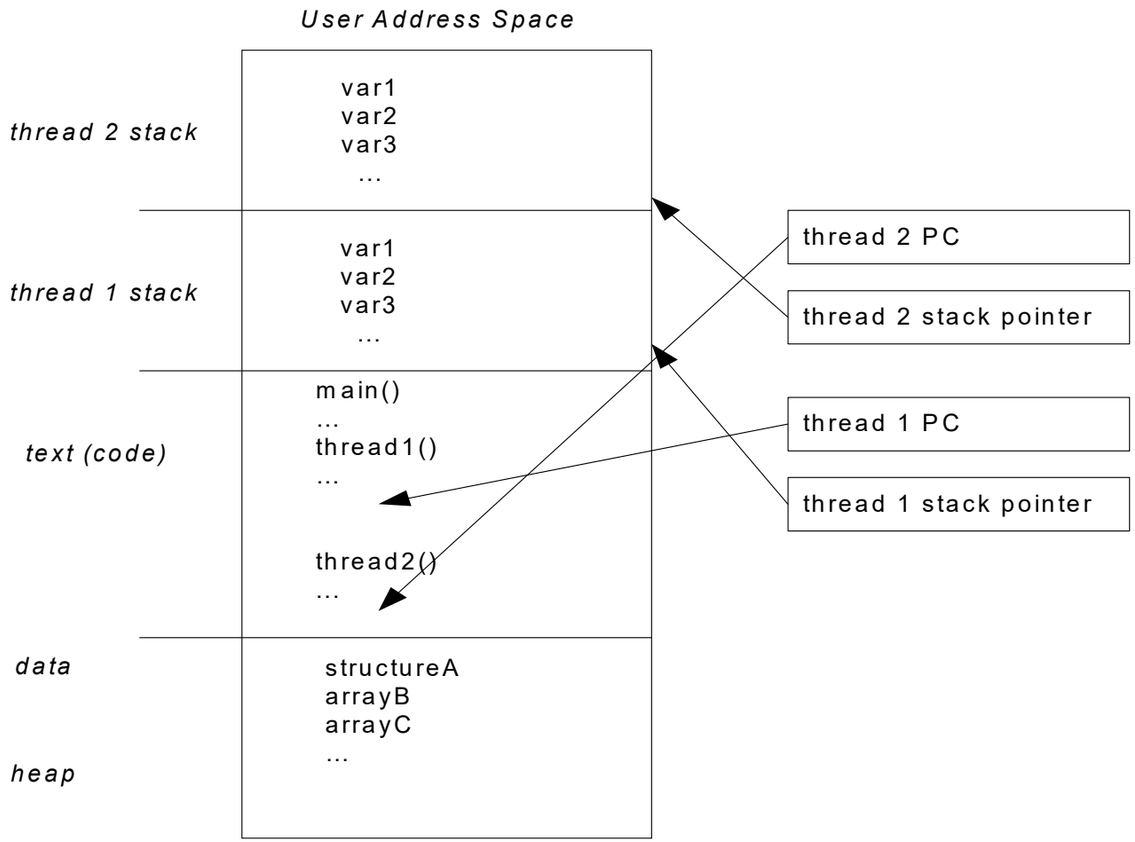


The "Context Switch Gap" (not a real term I made this up)

- 100,000 instructions + many DRAM accesses + loss of warmed state
  - Not viable for hiding even long-latency things like cache misses
  - Only viable for hiding "millenia" like disk access, blocked on conditional variable
- But, traditional multithreading (via pthreads, etc…) requires OS context switch
  - This effectively limits the space of programs that benefit from parallelism to a small set
  - Only the set with enough work to amortize the context switch overhead over
- ***As more CPU performance is reliant on longer warmup, overhead gets worse***
  - Hence "hardware multithreading" to hide shorter-latency events like cache misses
  - Cover this next time

Userspace Threads

- User programs can implement more abstractions on top of OS thread
- Runtime creates threads by allocating stack space
- Runtime switches threads via user level instructions (jump)
- But… **OS is not aware of these**
  - Cannot be harvested for TLP to hide long-latency events
  - OS cannot context switch between user space threads on long-latency events



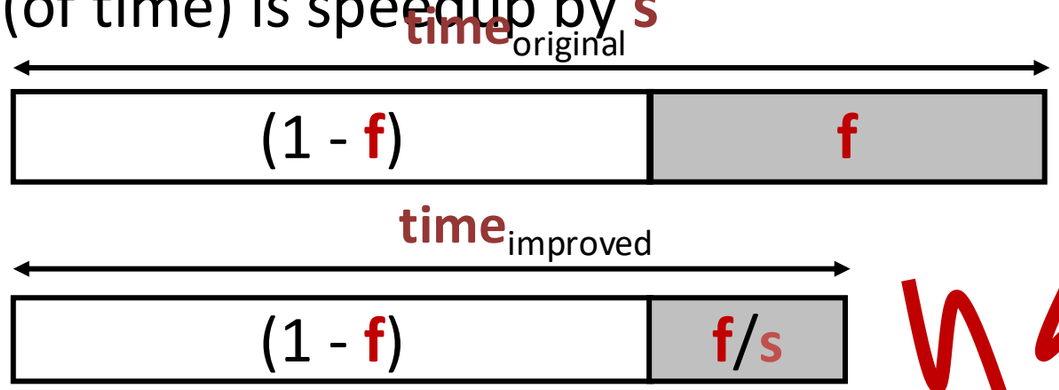
Shared-Memory Parallel Programming



Electrical & Computer ENGINEERING

Synchronization can easily be a bottleneck

- If only a fraction **f** (of time) is speedup by **s**



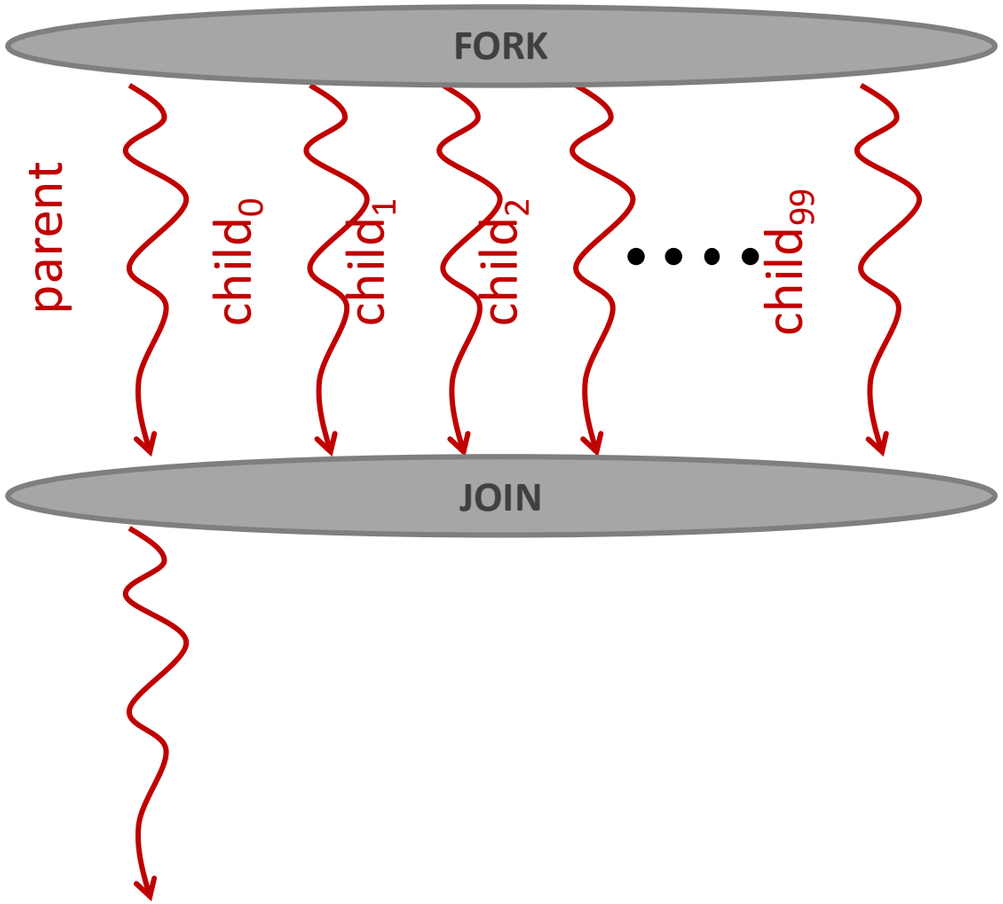
$$time_{improved} = time_{original} \cdot ((1-f) + f/s)$$

$$S_{effective} = 1 / ((1-f) + f/s)$$

- if **f** is small, s
- even when **f**

Borrowed with permission from James C Hoe

Synchronization can easily be a bottleneck



Borrowed with permission from James C Hoe

Shared memory synchronization

- A quick implementation walkthrough of a ***mutex***
  - ***I.e. a shared-memory mutual-exclusion algorithm***
- Exotic synchronization primitives exist
  - Thread barriers
  - Conditional Variables
  - Semaphores
  - Reader/Writer locks
- Massive distinction between
  - Spin-waiting (“unlocked soon by another core in parallel”)
  - Yielding remainder of time quanta via syscall (“soon by another thread on this core”)
  - Voluntary descheduling via syscall (”unlocked not anytime soon”)

Shared memory synchronization (attempt #1)

- With normal loads and stores

Try to replace this example

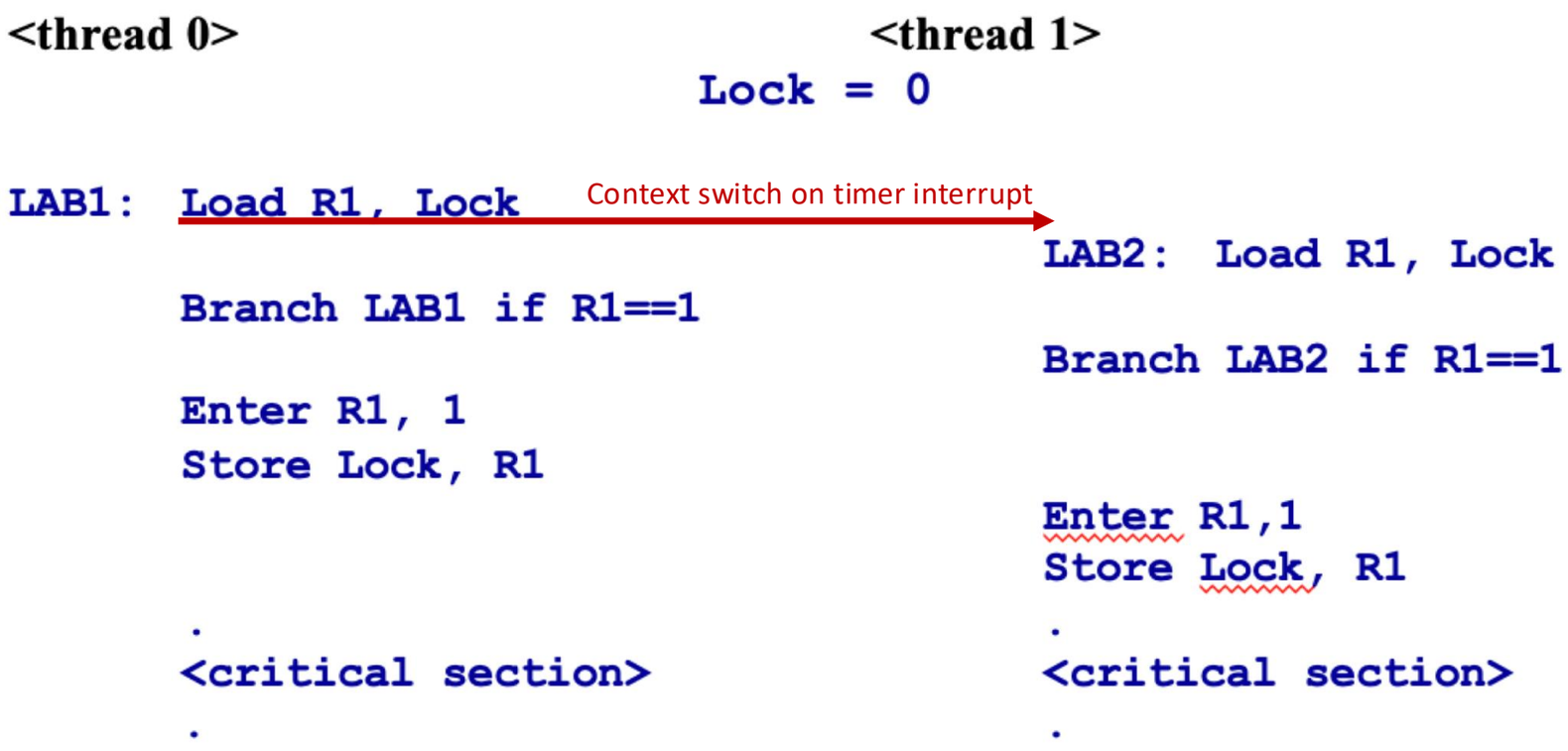
```
<thread 0>
        .
        .
LAB1:   Load R1, Lock
        Branch LAB1 if R1==1
        Enter R1, 1
        Store Lock, R1
        .
        <critical section>
        .
        Enter R1, 0
        Store Lock, R1
```

```
<thread 1>
        .
        .
LAB2:   Load R1, Lock
        Branch LAB2 if R1==1
        Enter R1,1
        Store Lock, R1
        .
        <critical section>
        .
        Enter R1, 0
        Store Lock, R1
```

Using the “spin-wait” flavor
Does not make sense on uni-processor!

Shared memory synchronization (attempt #1)

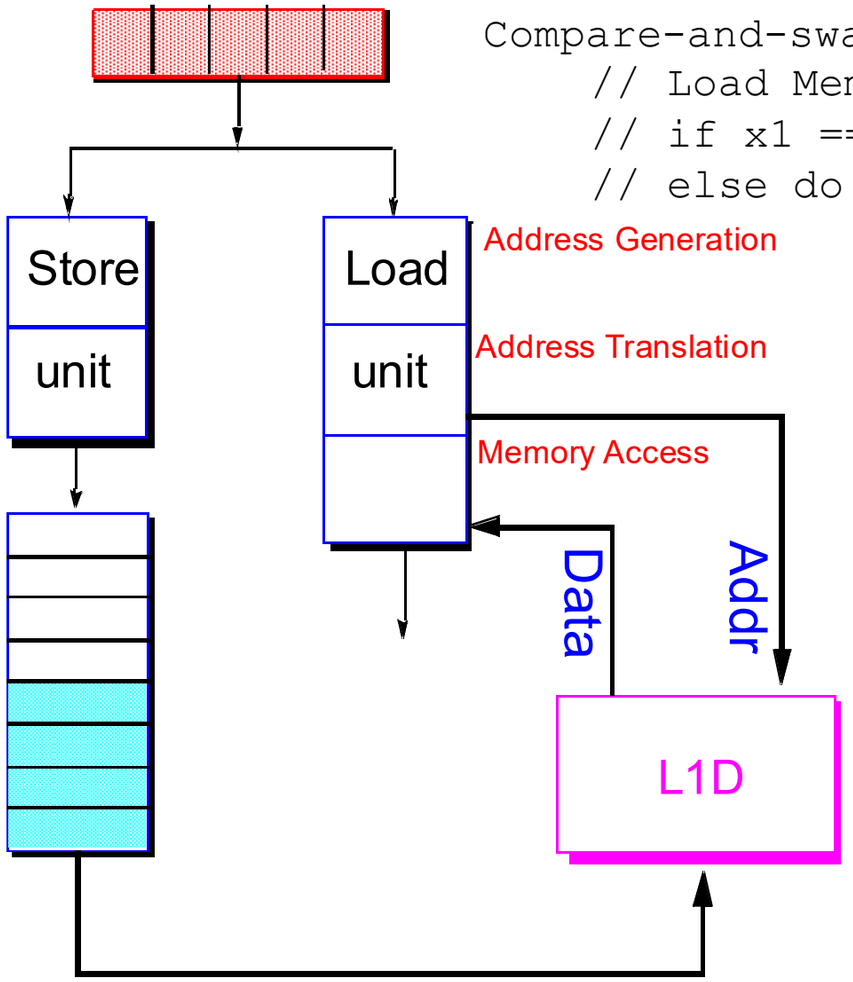
- Which are completely preemptible



Two main problems

- **On single-core** (and multi-core) **systems**
  - All user code is preemptible by timer interrupt
  - Must analyze code assuming a context switch can occur at any Macro-op boundary
  - Common handling technique is to implement ***Atomic RMW instructions***
- **On multi-core systems**
  - Synchronization is occurring in shared memory
  - Performance-wise, cache coherence may be a significant bottleneck
  - Functionally, weak memory consistency may make this very difficult to get right
  - Common handling technique is to heavily use ***Memory Barrier instructions***

Atomic RMW instruction




```
Compare-and-swap x1, x2, Mem[x3]
    // Load Mem[x3] into x1
    // if x1 == x2, store old value of x1 into Mem[x3]
    // else do nothing
```

Conceptually can be broken out into multiple uOps

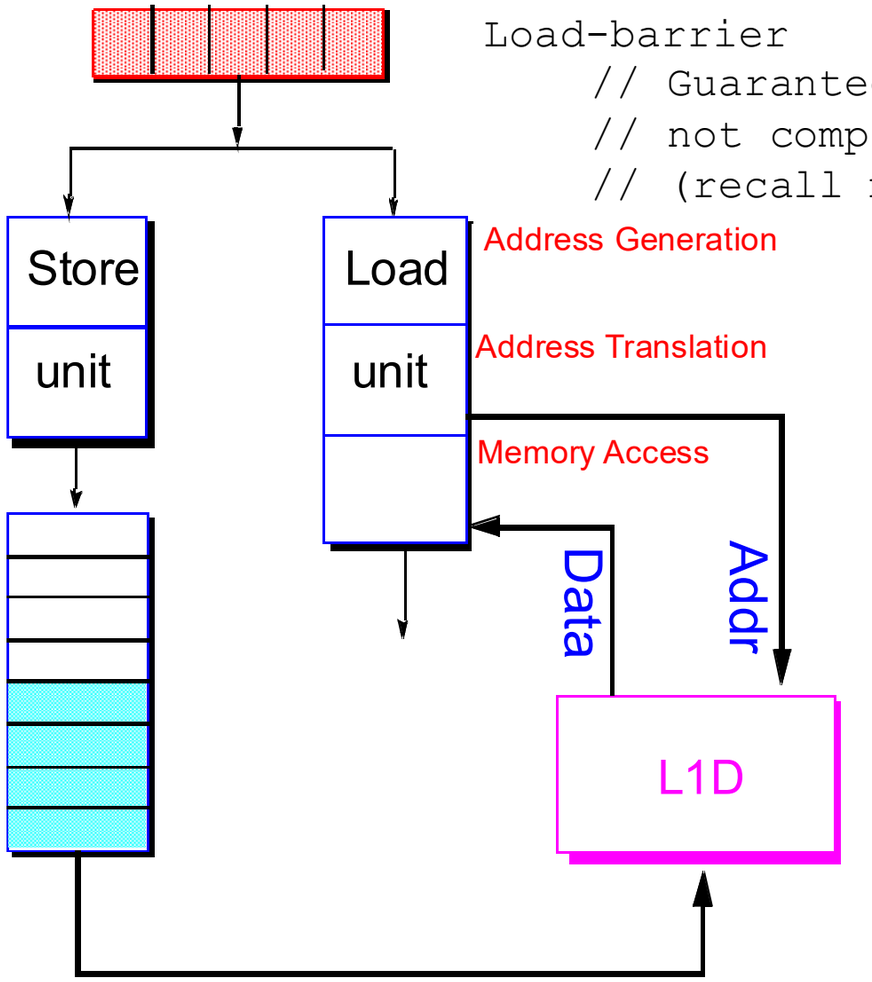
- Load operation
- Compare operation
- Store operation

When to request the cacheline? In what MSI/E/O state?
What to do if a snoop to the line comes in before the store is committed?

Simple solution – lock down memory system and block all snoops (probably wait until oldest or non-spec)

Better solutions?

Memory Barrier instruction




```
Load-barrier
    // Guarantees all younger loads
    // not completed until all older ops globally-visible
    // (recall much fancier versions exist)
```

When must we flush the Load/Store queue?
Is reordering around the barrier possible?
Is performance critical?

depends on frequency… which depends on model…

Case Study : Arm Break-before-Make

The hoops one must jump through to update a page table entry



Electrical & Computer ENGINEERING

Something like this (not exactly but close)

- Invalidate prior PTE (the break part)
- DSB – make sure the invalidation is seen by everyone
- ISB – make sure the DSB is executed first
- Write new PTE
- DSB – make sure the invalidation is seen by everyone

Page Table walker is usually not a coherent agent! This is even required on a single-core system

Something like this (not exactly but close)

- Invalidate prior PTE (the break part) — Send a broadcast TLB invalidation snoop to all agents
- DSB – make sure the invalidation is seen by everyone
- ISB – make sure the DSB is executed first
- Write new PTE
- DSB – make sure the invalidation is seen by everyone

Page Table walker is usually not a coherent agent! This is even required on a single-core system

Something like this (not exactly but close)

- Invalidate prior PTE (the break part)

MUST make sure TLB invalidate is seen and acked by all agents

- DSB – make sure the invalidation is seen by everyone
- ISB – make sure the DSB is executed first
- Write new PTE
- DSB – make sure the invalidation is seen by everyone

Page Table walker is usually not a coherent agent! This is even required on a single-core system

Something like this (not exactly but close)

Sync – start PTE update

- Invalidate prior PTE (the break part)
- DSB – make sure the invalidation is seen by everyone
- ISB – make sure the DSB is executed first
- Write new PTE
- DSB – make sure the invalidation is seen by everyone

Sync – end PTE update

If any other agent tries to access in the meantime they will get a translation failure,
Better than a functionally-incorrect wrong-mapping.

OS will catch the page fault and can check to see if we are in the middle of an update…
i.e. even more shared memory synchronization

Page Table walker is usually not a coherent agent! This is even required on a single-core system

Something like this (not exactly but close)

- Invalidate prior PTE (the break part)
- DSB – make sure the invalidation is seen by everyone
- ISB – make sure the DSB is executed first

Write new mapping

- Write new PTE
- DSB – make sure the invalidation is seen by everyone

Page Table walker is usually not a coherent agent! This is even required on a single-core system

Multicore Scheduling



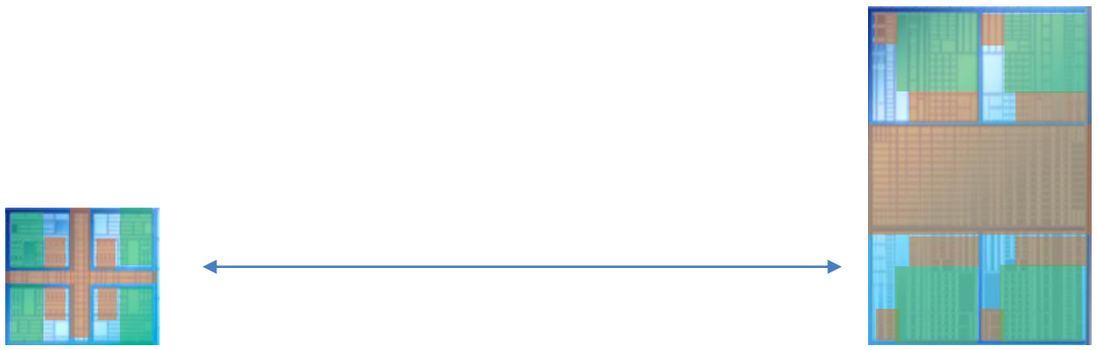
Electrical & Computer ENGINEERING

Scheduling algorithms are very important

- Guaranteed forward progress
  - Every thread must eventually make progress if not blocked
- Bounded waiting
  - The wait time should be bounded and reasonably “fair”
- General purpose scheduling algorithms
  - Optimized for fairness (FIFO) or overall system throughput
- Real-time scheduling algorithms
  - Hard deadlines may exist for getting certain things done (graphics render, engine throttle)
  - Certain threads may have higher priority than others

And where you put stuff matters

- Processor affinity
  - Attempt to keep a thread on the same core (for structure warmup)
- Heterogeneous core type
  - May try to schedule compute-intensive thread on compute-capable cores (and caches)
  - May try to schedule low-intensity threads on power efficient cores (and caches)
- Physical distance between threads sharing data (or from the memory it is using)



		Compute Cluster Cores				Low Power Island Cores			
Compute Cluster Cores			24.8	24.65	26	54.35	55.05	54.7	54.3
		23.85		27.25	27.45	54.65	55.2	55.6	54.1
		25.25	27.05		28.2	55.2	55.15	55.3	55.2
		25	27.5	27.9		56	55.15	55.3	56
Low Power Island Cores		55.05	54.45	55.4	55		23.7	23.6	22.7
		54.55	54.85	55.5	55.35	22.75		23.7	23.2
		55.1	55.05	55.55	55.15	23.4	23.6		23.5
		55.15	55.1	54.9	56.3	22.9	22.95	23.9	

~55 ns on LNL , 3X better vs Prev Gen

HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

And where you put stuff matters

- Dark silicon
  - May wish to power down parts of the chip
  - May wish to apply DVFS
- Some threads may be using accelerators
  - GPU, etc…
- Some threads may be resident in memory
  - And some may be paged out to disk
  - Try to pre-page-in these threads instead of running immediately?

Should elaborate on this more

An extremely (and usually proprietary) topic

18-740: Modern Computer Architecture & Design

Lecture 12:
"Parallel Architecture & Threading Models"

Scott Mionis

October 2, 2025

Next Time ...

- From **Synthesis Lectures on Computer Architecture**:
- Core Reading:
  - Sec. 3,4,5,6: Nemirovsky et al, "Multithreading Architectures," 2012
  - Sec. 2,3: Aamodt et al, "General-Purpose Graphics Architectures", 2018

Electrical & Computer ENGINEERING